Adaptive Insulin Dosage Prediction Using Reinforcement Learning

MAJOR PROJECT REPORT submitted in partial fulfillment of the requirements for the Award of the Degree of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

By

Ashraf Baig Mohammad (228W1A12H2)

Sarayu Guntupalli (228W1A12F2)

Under the Guidance of

Dr. G. Kalyani

Associate Professor

DEPARTMENT OF INFORMATION TECHNOLOGY

V R SIDDHARTHA ENGINEERING COLLEGE

(AUTONOMOUS – AFFILIATED TO JNTU, KAKINADA)

Approved by AICTE & Accredited by NBA

KANURU, VIJAYAWADA-520007

ACADEMIC YEAR

(2025-26)

V R SIDDHARTHA ENGINEERING COLLEGE

(Affiliated to JNTUK: Kakinada, Approved by AICTE, Autonomous)

(An ISO certified and NBA accredited institution)

Kanuru, Vijayawada – 520007

CERTIFICATE

This is to certify that this project report titled "**Adaptive Insulin Dosage Prediction Using Reinforcement Learning**" is a bonafide record of work done by **MD. ASHRAF BAIG (228W1A12H2) and G. SARAYU (228W1A12F2)** under my guidance and supervision is submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Information Technology, **V R Siddhartha Engineering College** (Autonomous under JNTUK) during the year **2025-26.**

(Dr. G. Kalyani)	**(Dr. M. Suneetha)**
Associate Professor	Dean R&D, Professor &Head
Dept. of Information Technology	Dept. of Information Technology

EXTERNAL EXAMINER SIGNATURE

Date of examination:

DEPARTMENT OF INFORMATION TECHNOLOGY

VR SIDDHARTHA ENGINEERING COLLEGE

PROJECT SUMMARY

S.No	Item	Description
1	**Project Title**	**Adaptive Insulin Dosage Prediction Using Reinforcement Learning**
2	**Student Names & Numbers**	Ashraf Baig Mohammad - 228W1A12H2 Sarayu Guntupalli - 228W1A12F2
3	**Name of the Guide**	Dr G. Kalyani
4	**Research Group**	Reinforcement Learning
5	**Application Area**	Health Care / Smart Diabetes Management
6	**Aim of the Project**	Adaptive Closed-Loop Insulin Dosage Control Using Comparative Reinforcement Learning (PPO, TD3, ER-PPO) for Type-1 Diabetes Management
7	**Project Outcomes**	• Closed-Loop Glucose Control (ER-PPO: 81.60% TIR) • Hypoglycemia Safety (1.50% Hypo Rate) • Comparative Algorithm Analysis (PPO / TD3 / ER-PPO)

Student Signatures:

1.
2.

Signature of the Guide

ACKNOWLEDGEMENT

First and foremost, I sincerely salute our esteemed institution **VR SIDDHARTHA ENGINEERING COLLEGE** for giving me this opportunity for fulfilling my project. I am grateful to our principal **Dr. A.V.RATNA PRASAD**, for his encouragement and support all through the way of my project.

On the submission of this Project report, I would like to extend my honour to **Dr.M.Suneetha, Dean R&D, Professor & Head of the Department, IT** for her constant motivation and support during the course of my work.

I feel glad to express my deep sense of gratefulness to my project guide **Dr.G.Kalyani**, Associate Professor for her guidance and assistance in completing this project successfully.

I would also like to convey my sincere indebtedness to all faculty members, including supporting staff of the Department, friends and family members who bestowed their great effort and guidance at appropriate times without which it would have been very difficult on my part to finish the project work.

TABLE OF CONTENTS

ACKNOWLEDGEMENT **iii**

LIST OF FIGURES **v**

LIST OF TABLES **vi**

ABSTRACT **vii**

CHAPTER-1 INTRODUCTION **1**

1.1 Origin of the Problem **2**

1.2 Basic definitions and Background **2**

1.3 Problem Statement with Objectives and Outcomes **6**

1.4 Related Applications **7**

CHAPTER-2 REVIEW OF LITERATURE **9**

2.1 Description of Existing Systems **9**

2.2 Summary of Literature Study **10**

2.3 Software Requirement Specification **22**

CHAPTER-3 PROPOSED METHOD **27**

3.1 Design Methodology **27**

3.2 System Architecture **30**

3.3 Description of Algorithms **25**

3.4 RL Environment Design, Dataset and Tools **42**

CHAPTER-4 RESULTS AND OBSERVATIONS **47**

4.1 Stepwise Description of Results **47**

4.2 Result Analysis **55**

4.3 Observations from the Work **56**

CHAPTER-5 CONCLUSION **59**

5.1 Conclusion **59**

5.2 Future Study **60**

REFERENCES

APPENDIX

LIST OF FIGURES

Figure Name	Page
Figure 3.1: Architecture for Phase 2 (PPO)	**31**
Figure 3.2: Closed-Loop RL Control Architecture for Phase 3 (ER-PPO)	**32**
Figure 3.3: Closed-Loop RL Control Architecture for Phase 3 (TD3)	**33**
Figure 4.1: PPO Agent — Terminal Evaluation Output	**51**
Figure 4.2: PPO Agent — Blood Glucose and Insulin Rate Trajectory(adult#001)	**51**
Figure 4.3: TD3 Agent — Terminal Evaluation Output	**52**
Figure 4.4: TD3 Agent — Blood Glucose and Insulin Rate Trajectory(adult#001)	**53**
Figure 4.5: ER-PPO Agent — Terminal Evaluation Output	**54**
Figure 4.6: ER-PPO Agent — Blood Glucose and Insulin Rate Trajectory(adult#001)	**54**

LIST OF TABLES

Table Name	Page
Table 2.1: Summary of Reviewed Reinforcement Learning-Based Systems	**10**
Table 3.1: State Space Comparison — ER-PPO vs TD3	**27**
Table 3.2: ER-PPO Reward Function Design	**28**
Table 3.3: Development Environment and Tools	**29**
Table 4.1: Phase 1 — Model Performance on AIM-94 Test Set	**31**
Table 4.2: Phase 2 — PPO Baseline Under Deterministic Meal Conditions	**32**
Table 4.3: Phase 3 — Comparative Results: PPO vs TD3 vs ER-PPO (Stochastic Meals)	**33**
Table 4.4: Cross-Phase Performance Summary (* Controlled meal scenario)	**38**

ABSTRACT

Managing insulin in Type-1 diabetes is difficult because every patient responds differently and glucose levels keep changing throughout the day. Traditional methods that use fixed insulin schedules or simple prediction models cannot adapt in real time to what the patient actually needs. This project builds a system that can control insulin dosage automatically by learning from the patient's own glucose patterns.

In the first phase (Mini Project 1), three deep learning models — LSTM, BiLSTM, and Temporal Fusion Transformer (TFT) — were trained on data from 70 Type-1 diabetic patients in the AIM-94 dataset. The goal was to predict the insulin dose needed in the next hour using the past 24 hours of data. TFT gave the best result with a Mean Absolute Error of 0.3954 U. However, this approach only predicts doses without actually adjusting them based on feedback, which is a major limitation.

In the second phase (Mini Project 2), a Proximal Policy Optimization (PPO) agent was trained inside the UVA/Padova Type-1 Diabetes simulator, which is FDA-accepted and used for testing insulin delivery systems. The PPO agent kept glucose within the safe range 86.55% of the time under fixed meal conditions. But when meals were given at random times and amounts, the agent did not perform well, which led to the main project work.

The major project (Module 3) compared three reinforcement learning algorithms - standard PPO, Twin Delayed Deep Deterministic Policy Gradient (TD3), and the proposed Entropy-Regulated PPO (ER-PPO) — under random meal conditions. ER-PPO works by starting with more exploration and slowly reducing it as training goes on. This helps the agent find safe insulin doses early and stick to them later. ER-PPO achieved a Time-in-Range of 81.60% with only 1.50% hypoglycemia, which is the only result to meet the clinical target of 75% TIR. TD3 had a dangerously high hypoglycemia rate of 35.70% and is not clinically safe. These results show that adaptive entropy control in reinforcement learning is a good approach for building safe and effective insulin dosage systems for Type-1 Diabetes patients.

Keywords: insulin dosage prediction, reinforcement learning, PPO, TD3, ER-PPO, entropy regulation, Type-1 diabetes, closed-loop control, Time-in-Range, simglucose, UVA/Padova simulator, deep learning

CHAPTER-1

Introduction

1.1 Origin of the Problem

Diabetes is one of the most common and serious health problems in the world today. As of 2021, more than 537 million adults were living with diabetes, and this number is expected to reach 783 million by 2045. India has the second-largest diabetic population in the world, which makes this a very important issue for our country as well.

The biggest challenge in managing Type-1 diabetes is not just giving insulin, but giving the right amount at the right time. A healthy pancreas does this automatically by continuously monitoring blood glucose levels and adjusting insulin secretion. But for a diabetic patient, this process has to be done manually or through external systems, which is not easy to maintain accurately in real-life situations [1].

There are many factors that affect blood glucose levels such as food intake, physical activity, stress, illness, and even sleep patterns. Because of this, insulin requirements are not constant and keep changing throughout the day. This makes it difficult for patients to decide how much insulin to take, and small mistakes can lead to serious conditions like hypoglycemia or hyperglycemia.

Traditional methods like fixed insulin schedules and basal-bolus therapy depend heavily on doctor consultations and patient self-reporting. These methods are not only time-consuming but also prone to human error. Patients may forget to log meals properly or miscalculate insulin doses. As a result, glucose control is often inconsistent.

In recent years, technologies like Continuous Glucose Monitoring (CGM) sensors and insulin pumps have improved diabetes management. These devices provide real-time glucose readings and allow more precise insulin delivery. However, most of these systems still require manual input or decision-making from the patient, which limits their effectiveness [2].

Because of these challenges, there is a growing need for intelligent automated systems that can monitor glucose levels continuously and make insulin decisions without

human intervention. This is where machine learning and especially reinforcement learning can play an important role.

This project addresses this problem in a structured way through three modules. In Module 1, deep learning models such as LSTM, BiLSTM, and Temporal Fusion Transformer (TFT) were trained to predict the next-hour insulin dose using patient data from the AIM-94 dataset. This stage helps in understanding the patterns in glucose and insulin data, but it does not include feedback from real-time conditions [3].

In Module 2, the system is extended to a closed-loop setup using reinforcement learning. A PPO-based agent was trained in the UVA/Padova simulator and achieved a Time-in-Range of 86.55% under fixed meal conditions. This shows that RL can learn to control insulin delivery effectively, but the environment is still simplified and predictable.

Finally, in Module 3, which is the main focus of this project, a more realistic setup is used with random meal disturbances. Three algorithms — PPO, TD3, and ER-PPO — were compared under the same conditions. Among them, ER-PPO achieved the best performance in terms of both accuracy and safety, mainly due to its adaptive entropy mechanism [4].

Overall, this work tries to move from simple prediction to intelligent control, and finally towards a more reliable and safe automated insulin delivery system.

1.2 Basic definitions and Background

This section explains the key terms and background ideas needed to understand this project. These concepts are important because the whole system is based on them, so having a clear idea helps in understanding later parts.

1.2.1 Diabetes Mellitus

Diabetes mellitus is a condition where the body cannot properly regulate blood glucose levels. In normal conditions, the pancreas produces insulin which helps glucose enter the cells.

In Type-1 Diabetes (T1DM), the immune system attacks and destroys the insulin-producing beta cells in the pancreas. Because of this, the body cannot produce insulin

at all. Patients with T1DM need to take insulin externally for their entire life, either through injections or insulin pumps [5].

Managing T1DM is difficult because glucose levels keep changing due to many daily factors, so insulin has to be adjusted continuously.

1.2.2 Insulin Dosage Prediction

Insulin dosage prediction refers to estimating how much insulin a patient should take at a particular time using computational models.

In Module 1 of this project, this is treated as a regression problem. The model takes input from the past 24 hours (like glucose levels, meals, insulin history) and predicts the insulin dose required for the next hour [6].

This helps in understanding patterns in the data, but it does not consider how the body actually reacts after giving insulin.

1.2.3 Reinforcement Learning

Reinforcement Learning (RL) is a type of machine learning where an agent learns by interacting with an environment.

The agent takes an action, observes the result, and receives a reward or penalty. Over time, it learns which actions give better outcomes.

RL is especially useful in problems where decisions need to be made continuously, like insulin dosing, because the environment (human body) keeps changing [7].

1.2.4 Markov Decision Process (MDP)

Most RL problems are modeled as a Markov Decision Process (MDP).

An MDP consists of:

- State space (S) — current condition of the system
- Action space (A) — possible actions the agent can take
- Transition probability (P) — how the system moves from one state to another
- Reward function (R) — feedback given to the agent
- Discount factor (γ) — importance of future rewards

These components together define how the agent learns and improves its decisions over time.

1.2.5 Policy and Value Function

A policy defines how the agent chooses actions based on the current state.

It can be:

- Deterministic → always gives the same action
- Stochastic → gives a probability distribution of actions

The value function estimates how good a state is, based on expected future rewards. It helps the agent understand long-term benefits rather than just immediate rewards.

1.2.6 Reward Function

The reward function is one of the most important parts of RL because it directly guides the learning process.

In this project, the ER-PPO agent receives:

- +1.0 when glucose is within the safe range (70–180 mg/dL)
- -0.5 for small deviations
- -2.0 for dangerous conditions (below 54 or above 250 mg/dL)

Additionally, a smoothness aspect is considered so that the model avoids sudden large insulin changes. If the reward is not designed properly, the model may learn unsafe behaviour, so this part is very critical.

1.2.7 Time-in-Range, Hypoglycemia and Hyperglycemia

Time-in-Range (TIR) is the main performance metric used in this project.

It measures the percentage of time the glucose level stays within the safe range (70–180 mg/dL). For Type-1 diabetic patients, the clinical target is at least 75% TIR.

- Hypoglycemia → glucose below 70 mg/dL (dangerous)
- Hyperglycemia → glucose above 180 mg/dL

Maintaining a balance is important because both conditions are harmful if they occur frequently [8].

1.2.8 Entropy in Reinforcement Learning

Entropy in RL represents how random or uncertain the agent's decisions are.

- High entropy → more exploration (trying different actions)
- Low entropy → more stable decisions (exploitation)

In ER-PPO, entropy starts at 0.05 and gradually reduces to 0.001 during training. This helps the agent explore different strategies initially and then become more stable later.

If entropy is not controlled properly, the model may either become too random or too rigid.

1.2.9 Closed-Loop Insulin Delivery and Artificial Pancreas

A closed-loop insulin delivery system, also known as an Artificial Pancreas, automatically monitors glucose levels and adjusts insulin without manual input.

It mainly consists of:

- CGM sensor (to measure glucose)
- Insulin pump (to deliver insulin)
- Control algorithm (to decide dosage)

This project focuses on developing the control algorithm part. Instead of testing on real patients, the system is evaluated using a simulator.

1.2.10 UVA/Padova Type-1 Diabetes Simulator (simglucose)

The UVA/Padova simulator is a mathematical model that represents glucose-insulin dynamics in the human body.

It is accepted by the U.S. FDA for testing insulin control systems, which makes it reliable for research purposes. In Python, it is available through the simglucose library.

The simulator includes 30 virtual patients:

- 10 adults

- 10 adolescents
- 10 children

In this project, it is used in Module 2 and Module 3 to train and evaluate RL models under safe and controlled conditions [9].

1.3 Problem Statement with Objectives and Outcomes

1.3.1 Problem Statement

Patients with Type-1 diabetes need a system that can control insulin in real time, handle differences between patients, and keep glucose levels safe without causing hypoglycemia. Current supervised learning models can only predict doses but cannot adjust them based on what actually happens after the dose. Existing RL-based systems either use fixed exploration settings that cause instability, or use algorithms like TD3 that produce unsafe dosing without proper constraints.

1.3.2 Objectives

The primary objectives of this project are:

- Process and prepare patient time-series data from the AIM-94 dataset and train deep learning models for next-hour insulin dose prediction.
- Train a PPO-based RL agent inside the UVA/Padova simulator for closed-loop blood glucose control and establish a TIR baseline.
- Design and implement the ER-PPO algorithm with adaptive entropy decay, and show that it meets clinical TIR and safety targets under random meal conditions.
- Compare all three project phases and all three RL algorithms using clinical metrics including TIR, hypoglycemia rate, hyperglycemia rate, mean glucose, and total insulin delivered.

1.3.3 Outcomes

The proposed system is expected to have these outcomes:

- A trained set of deep learning models (LSTM, BiLSTM, TFT) for insulin dose prediction, with TFT achieving the lowest error of 0.3954 U MAE on the AIM-94 test set.

- A PPO-based RL agent trained in the UVA/Padova simulator achieving 86.55% Time-in-Range under a controlled meal scenario.
- The ER-PPO algorithm achieving 81.60% TIR and only 1.50% hypoglycemia under random meal conditions, the only algorithm in this study to meet both clinical benchmarks.
- A full comparison of PPO, TD3, and ER-PPO showing ER-PPO is the safest and most effective algorithm for closed-loop insulin control.

1.4 Related Applications

The system developed in this project can be applied in several real-world scenarios, especially in healthcare and smart medical systems. Since it focuses on automated insulin control using reinforcement learning, its usage is not limited to just one area.

• **Artificial Pancreas Systems:** The RL-based control method can be directly integrated into artificial pancreas devices. These systems combine CGM sensors and insulin pumps, and the model can automatically decide insulin doses without manual intervention. This can reduce patient effort and improve glucose stability.

• **Clinical Decision Support:** Doctors and endocrinologists can use the results from PPO, TD3, and ER-PPO comparison to understand which algorithm performs better under different conditions. It can help in selecting safer and more reliable methods for automated insulin therapy.

• **Wearable and IoT Devices:** The model can be deployed on wearable insulin pumps connected through IoT. Since the model is relatively lightweight, it can run in real-time and continuously adjust insulin delivery based on incoming sensor data [10]. This makes it suitable for modern smart healthcare devices.

• **Mobile Health Applications:** The trained model can be used as a backend for mobile applications. Patients can get insulin dose recommendations based on their current glucose readings, which can help in better daily management. It also reduces dependency on manual calculations.

• **ICU Glucose Management:** The same closed-loop control idea can be applied in intensive care units (ICU), where patients often have unstable glucose levels. It can

replace traditional sliding-scale insulin methods, which are less adaptive and more manual [11].

• **Remote Patient Monitoring:** In rural or remote areas where access to specialists is limited, such systems can provide continuous monitoring and insulin adjustment. This can improve patient care without requiring frequent hospital visits.

• **Pharmaceutical Research:** The UVA/Padova simulator used in this project can also support pharmaceutical research. It allows testing of new insulin delivery strategies or drug responses in a safe virtual environment before moving to real clinical trials [12].

CHAPTER-2

Review of Literature

2.1 Description of Existing Systems

Research in automated glucose management and insulin dosage estimation can be broadly divided into two main categories: supervised deep learning models for open-loop prediction, and reinforcement learning-based systems for closed-loop control. Early work in this area mainly focused on recurrent neural networks, since patient physiological data is naturally sequential and time-dependent.

Beauchamp et al. developed a combined LSTM and deep residual network architecture for simultaneous prediction of carbohydrate intake and insulin dosage requirements. Their work showed that multi-output models can capture the relationship between food intake and insulin needs, which is actually important in real-world scenarios.

Kozinetz et al. compared classical machine learning models such as Random Forest and Gradient Boosting with deep learning approaches like MLP and LSTM. Their results showed that deep learning models generally perform better when the data is complex and high-dimensional, which is usually the case in medical datasets.

Klonoff et al. applied the Temporal Fusion Transformer (TFT) to predict insulin-on-board levels. The attention mechanism in TFT helps the model focus on important time periods rather than treating all past data equally, which improves prediction quality [13].

However, a common limitation across all these supervised approaches is that they are open-loop. Once the model predicts an insulin dose, it cannot adjust based on what actually happens next in the patient's body. This is a major drawback because glucose levels can change rapidly due to many factors.

Reinforcement learning addresses this limitation by introducing a feedback loop, where the agent continuously observes the patient's state and adjusts insulin accordingly. This allows real-time adaptation, which is more suitable for clinical use [14].

2.2 Summary of Literature Study

The following six studies constitute the primary body of prior art in reinforcement learning applied to insulin dosage control and glycemic management. Each is examined with respect to methodology, results, and limitations pertinent to the proposed ER-PPO framework.

2.2.1 A Dual Mode Adaptive Basal-Bolus Advisor Based on Reinforcement Learning

Authors: Q. Sun, M. N. Jankovic, L. Bally, J. M. Mougiakakou

Year: 2022

Journal: IEEE Journal of Biomedical and Health Informatics, vol. 26, no. 1, pp. 436-445

Sun et al. proposed a dual-mode RL advisor for basal-bolus insulin dosing. The system alternates between a basal mode that continuously modulates background insulin delivery and a bolus mode that computes meal-time correction doses. The RL agent was trained using a reward function penalising both hyperglycaemic and hypoglycaemic deviations from the target range, and the dual-mode architecture addresses the common limitation of systems that optimise only one dosing component [15].

This approach is useful because in real-life insulin therapy, both basal and bolus insulin are required, and handling them together makes the system more practical. The separation of modes also simplifies the learning process to some extent.

Despite its methodological contribution, the system was validated exclusively in simulation under fixed-dose meal scenarios. Crucially, the entropy coefficient of the RL policy was held constant throughout the training process. A static exploration parameter prevents the policy from transitioning from broad early exploration to stable late-stage exploitation, which can compromise the safety and consistency of dosing behaviour in final deployment. This limitation directly motivates the adaptive entropy decay mechanism central to ER-PPO [15].

2.2.2 Optimized Glycemic Control of Type 2 Diabetes with Reinforcement Learning

Authors: T. Zhu, K. Li, P. Herrero, P. Georgiou

Year: 2023

Journal: Nature Medicine, vol. 29, pp. 1670-1680

Zhu et al. conducted a proof-of-concept clinical trial in which an RL agent was trained to recommend adjustments to oral medication dosing for Type-2 Diabetes patients, using patient-reported glucose measurements, dietary information, and activity data as inputs. The study demonstrated statistically significant reductions in HbA1c relative to standard care protocols, representing a meaningful clinical validation of RL-based glycemic management [16].

This work is important because it shows that reinforcement learning can be applied successfully in real clinical settings, not just simulations. It also highlights how patient lifestyle data can be integrated into decision-making.

The system is architecturally designed for Type-2 Diabetes and operates over a discrete, long-horizon action space, recommending daily medication adjustments rather than continuous real-time insulin delivery. This is fundamentally incompatible with the second-to-second closed-loop control requirements of Type-1 Diabetes management that constitute the focus of the present project [16].

2.2.3 Personalized Insulin Dosing Using RL for High-Fat Meals and Exercise in T1D

Authors: I. Contreras, J. M. Revert, J. Vehi

Year: 2024

Journal: Nature Communications, vol. 15

Contreras et al. presented a personalized RL approach for Type-1 Diabetes in which a separate agent was trained for each patient using retrospective CGM and insulin records, with reward functions individually tuned to each patient's glycemic response profile. The system was evaluated under high-fat meals and exercise conditions - two of the most physiologically challenging disturbances for closed-loop control - and

demonstrated significant reductions in both postprandial hyperglycaemia and exercise-induced hypoglycaemia [17].

This approach shows that personalization can significantly improve performance, especially under complex real-world scenarios like exercise and high-fat meals.

However, the per-patient training requirement constitutes a fundamental scalability limitation. Each new patient demands a full and independent training cycle, rendering the approach computationally prohibitive for large-scale clinical deployment. The ER-PPO system developed in this project is trained once and evaluated across multiple virtual patients without patient-specific adaptation [17].

2.2.4 Deep Reinforcement Learning for Personalized Insulin Dosing in ICU Patients

Authors: Y. Chen, X. Lu, Z. Zhang

Year: 2025

Journal: International Journal of Data Science and Analytics

Chen et al. applied deep RL to insulin dosing in the intensive care unit, a clinical setting characterised by elevated hyperglycaemia risk due to stress-induced hormonal changes and nutritional interventions. The agent was trained on retrospective ICU patient data and benchmarked against conventional sliding-scale infusion protocols, achieving superior time-in-range performance in retrospective evaluation [18].

This work highlights the applicability of RL in critical care environments where glucose levels are highly unstable and difficult to manage manually.

A significant methodological limitation is the use of a fixed entropy coefficient throughout training, which produces inconsistent and unstable dosing behaviour in late-stage training simulations. This observation reinforces the clinical necessity of adaptive entropy scheduling: without a mechanism to reduce policy randomness as training matures, the agent retains exploration-driven variability at deployment, posing an unacceptable safety risk in critical care settings [18].

2.2.5 Deep Reinforcement Learning for Closed-Loop Blood Glucose Control in Type 1 Diabetes

Authors: A. Fox, J. W. Lee, D. J. Shields, T. T. Bhatt

Year: 2020

Journal: arXiv preprint arXiv:2009.09051

Fox et al. conducted one of the earliest systematic investigations of deep RL for closed-loop glucose control in Type-1 Diabetes using the UVA/Padova simulator. The study compared three algorithms — Deep Deterministic Policy Gradient (DDPG), Proximal Policy Optimization (PPO), and Soft Actor-Critic (SAC) — across multiple virtual patient profiles in the simglucose environment [19].

The results indicated that stochastic on-policy methods, particularly SAC, achieved superior glycemic control relative to deterministic off-policy approaches under meal disturbance conditions. This suggests that controlled randomness can actually improve performance in uncertain environments.

However, all three algorithms employed fixed entropy coefficients, and none investigated the clinical consequences of adaptive entropy regulation during training. The absence of TD3 from the comparison also leaves a methodological gap regarding the relative suitability of on-policy versus off-policy algorithms under identical stochastic conditions — a gap addressed directly by the Phase 3 comparative study in this project [19].

2.2.6 A Reinforcement Learning-Based Method for Management of Type 1 Diabetes

Authors: S. M. Pappada, B. D. Cameron, P. M. Rosman, R. E. Bourey

Year: 2019

Journal: JMIR Diabetes, vol. 4, no. 3, e12905

Pappada et al. presented one of the earliest direct applications of reinforcement learning to Type-1 Diabetes management, employing a tabular Q-learning agent operating on a discretized state and action space derived from continuous glucose

monitor readings and meal intake reports. The agent was evaluated retrospectively against a small cohort of T1D patients [20].

This work is important historically, as it demonstrates the initial feasibility of applying RL in diabetes management.

The authors acknowledged the fundamental limitation that discretizing a continuous glucose space introduces approximation error and prevents the fine-grained dosing resolution required for safe clinical deployment. The approach cannot scale to the high-dimensional state representations necessary for real-world performance. The deep RL algorithms evaluated in this project — PPO, TD3, and ER-PPO — operate entirely in continuous state and action spaces, directly addressing these architectural constraints [20].

2.2.7 Reinforcement Learning-Based Artificial Pancreas for Type 1 Diabetes Control

Authors: R. D. Nimri, N. Muller, M. Atlas, et al.

Year: 2021

Journal: Diabetes Technology & Therapeutics, vol. 23, no. 6

Nimri et al. developed a reinforcement learning-based artificial pancreas system designed for closed-loop insulin delivery in Type-1 Diabetes patients. The RL agent dynamically adjusted insulin infusion rates based on continuous glucose monitor (CGM) data, aiming to maximise time-in-range while avoiding hypoglycaemia. The system was validated in clinical trials involving outpatient scenarios, demonstrating improved glycaemic control compared to standard pump therapy [21].

This shows that RL-based systems can move beyond simulation and achieve real-world clinical validation, which is a significant step forward.

Despite clinical validation, the system relies on predefined safety constraints and conservative policy updates, which limit its adaptability to rapidly changing physiological conditions. Additionally, the exploration strategy is tightly restricted, preventing the agent from learning more optimal dosing strategies beyond clinician-defined boundaries. This highlights the need for controlled yet adaptive exploration mechanisms, such as entropy decay, to balance safety and learning efficiency [21].

2.2.8 Safe Reinforcement Learning for Blood Glucose Control in Type 1 Diabetes

Authors: M. S. Islam, H. M. Faisal, A. S. Rahman

Year: 2022

Journal: IEEE Access, vol. 10

Islam et al. proposed a safe reinforcement learning framework for insulin dosing that integrates risk-sensitive reward functions to explicitly penalise unsafe glucose excursions. The model incorporates safety layers that override RL actions when predicted glucose levels fall below critical thresholds, ensuring patient safety during policy execution. Simulation results using the UVA/Padova simulator demonstrated improved safety metrics compared to standard RL approaches [22].

While this improves safety, it introduces dependence on external rules.

However, the integration of rule-based safety overrides introduces a hybrid control dependency that limits the autonomy of the RL agent. The agent does not fully learn to avoid unsafe regions independently, as safety is externally enforced rather than internally optimised. This creates a dependency on heuristic constraints and restricts the generalisation capability of the learned policy under unseen conditions [22].

2.2.9 Deep Q-Network-Based Blood Glucose Control for Type 1 Diabetes

Authors: H. Lee, J. Kim, S. Park

Year: 2020

Journal: Computers in Biology and Medicine, vol. 123

Lee et al. introduced a Deep Q-Network (DQN)-based approach for automated insulin dosing in Type-1 Diabetes, where the continuous glucose state space was discretised and mapped to discrete insulin action levels. The system demonstrated improved glycaemic stability compared to baseline PID controllers in simulation environments, particularly under regular meal patterns [23].

This shows that RL can outperform traditional controllers, even in simpler setups.

However, the discretisation of both state and action spaces significantly limits the resolution of insulin control, making the approach unsuitable for fine-grained, real-time glucose regulation. Moreover, DQN's off-policy nature and instability in

continuous domains make it less appropriate for safety-critical applications like insulin dosing. These limitations motivate the use of continuous-action algorithms such as PPO and SAC in modern approaches [23].

2.2.10 Model-Based Reinforcement Learning for Artificial Pancreas Systems

Authors: C. Daskalaki, P. Diem, S. G. Mougiakakou

Year: 2021

Journal: IEEE Transactions on Biomedical Engineering, vol. 68, no. 12

Daskalaki et al. proposed a model-based reinforcement learning framework that integrates physiological models of glucose-insulin dynamics into the learning process. By leveraging prior knowledge of metabolic responses, the RL agent achieves faster convergence and improved sample efficiency compared to model-free approaches. The system demonstrated strong performance in simulation across multiple virtual patient profiles [24].

This approach is useful when data is limited, since prior knowledge helps guide learning.

However, reliance on predefined physiological models introduces model bias, which can degrade performance when real patient dynamics deviate from assumed conditions. The approach also lacks flexibility in adapting to unforeseen disturbances such as stress or illness. This underscores the advantage of model-free approaches like ER-PPO, which learn directly from interaction data without strong prior assumptions [24].

The **Table 2.1** provides a structured summary of the reviewed reinforcement learning-based systems for insulin dosage control, highlighting key aspects such as datasets used, algorithms applied, purpose of each study, and their respective limitations. It gives an overall comparison of different approaches considered in this work.

Table 2.1: Summary of Reviewed Reinforcement Learning-Based Systems

S.No	Title & Publication Details	Datasets Used	Algorithms / Technologies Used	Purpose	Remarks
1	A Dual Mode Adaptive Basal-Bolus Advisor Based on Reinforcement Learning [15] Sun et al., IEEE J. Biomed. Health Inform., 2022	UVA/Padova T1D simulator (simglucose)	Dual-mode RL; basal-bolus advisor; reward shaping	Improve TIR over rule-based basal-bolus control	Simulation only; fixed entropy coefficient; no stochastic meal evaluation
2	Optimized Glycemic Control of Type 2 Diabetes with Reinforcement Learning [16] Zhu et al., Nature Medicine, 2023	Real-world T2D patient clinical data	Deep RL; discrete action space; daily medication recommendation	Reduce HbA1c and fasting glucose in T2D patients	Designed for T2D; discrete daily actions; inapplicable to T1D closed-loop control
3	Personalized Insulin Dosing Using RL for	Retrospective T1D CGM and insulin	Personalized RL; per-patient reward function	Reduce postprandial hyperglycaemia	Requires per-patient retraining; not

	High-Fat Meals and Exercise in T1D [17] Contreras et al., Nature Communications, 2024	records	tuning	and exercise-induced hypoglycaemia	scalable for population-level deployment
4	Deep RL for Personalized Insulin Dosing in ICU Patients [18] Chen et al., Int. J. Data Sci. Anal., 2025	Retrospective ICU patient glucose records	Deep RL; retrospective ICU protocol comparison	Improve TIR over conventional ICU sliding-scale protocols	ICU-specific; retrospective evaluation; fixed entropy produces unstable late-stage dosing
5	Deep RL for Closed-Loop Blood Glucose Control in T1D [19] Fox et al., arXiv preprint, 2020	UVA/Padova T1D simulator (simglucose)	DDPG, PPO, SAC comparative evaluation on virtual patients	Compare deep RL algorithms for T1D closed-loop control	All algorithms use fixed entropy; no adaptive exploration; TD3 not evaluated
6	A RL-Based Method for Management of Type 1 Diabetes	Small retrospective T1D patient cohort	Tabular Q-learning; discretized state-action	Demonstrate RL feasibility for T1DM glucose management	Tabular formulation; discretized states; no deep

	[20] Pappada et al., JMIR Diabetes, 2019		space		learning; limited scalability
7	Reinforcement Learning-Based Artificial Pancreas for Type 1 Diabetes Control (Nimri et al., 2021, *Diabetes Technology & Therapeutics*) [21]	Clinical trial data + CGM readings + insulin pump data	Reinforcement Learning (closed-loop control), CGM integration, insulin pump system	To develop an RL-based artificial pancreas for real-time insulin delivery	To develop an RL-based artificial pancreas for real-time insulin delivery
8	Safe Reinforcement Learning for Blood Glucose Control in Type 1 Diabetes (Islam et al., 2022, *IEEE Access*) [22]	UVA/Padova simulator dataset (virtual patients)	Safe RL, risk-sensitive reward function, safety layer overrides	To ensure safe insulin dosing while minimizing hypo/hyperglycemia	To ensure safe insulin dosing while minimizing hypo/hyperglycemia
9	Deep Q-Network-Based	Simulated CGM data +	Deep Q-Network	To automate insulin dosing	To automate insulin dosing

	Blood Glucose Control for Type 1 Diabetes (Lee et al., 2020, *Computers in Biology and Medicine*) [23]	meal scenarios	(DQN), discrete state-action space	using value-based RL	using value-based RL
10	Model-Based Reinforcement Learning for Artificial Pancreas Systems (Daskalaki et al., 2021, *IEEE Transactions on Biomedical Engineering*) [24]	Simulated patient data (physiological models)	Model-Based RL, glucose-insulin physiological models	To improve sample efficiency and control performance	To improve sample efficiency and control performance

The reviewed literature, while collectively advancing the state of the art, exhibits several recurring limitations that the proposed ER-PPO system is specifically designed to address.

1. **Simulation-Only Evaluation:** Most of the reviewed systems were validated only in simulation environments without real patient testing. While simulators like UVA/Padova are reliable, they still cannot fully capture real-world variability.

Because of this, there is limited confidence in how these models will perform in actual clinical conditions.

2. **Fixed Entropy Coefficients:** A common limitation across many on-policy reinforcement learning systems is the use of a fixed entropy coefficient throughout training. This prevents a proper transition from exploration to stable learning. As a result, the model may continue to behave unpredictably even in later stages, leading to inconsistent and sometimes unsafe insulin dosing.

3. **Inapplicability to Type-1 Diabetes Dynamics:** The work by Zhu et al. focuses on Type-2 Diabetes and operates using discrete daily decisions. However, Type-1 Diabetes requires continuous, real-time insulin adjustments. So, even though the approach is effective in its own domain, it cannot be directly applied to T1D closed-loop control systems.

4. **Per-Patient Training Requirement:** The personalized RL approach proposed by Contreras et al. requires training a separate model for each patient. While this improves individual performance, it is not practical for large-scale deployment. Training and maintaining separate models for many patients would be computationally expensive and difficult to manage.

5. **Incomplete State Representation:** Some early RL-based systems, such as the one by Pappada et al., used only blood glucose levels as input. This is not sufficient for proper decision-making. Important features like rate of change of glucose (ΔBG), insulin-on-board (IOB), and carbohydrates-on-board (COB) are missing, which limits the model's ability to anticipate future glucose trends.

6. **Absence of Multi-Algorithm Comparative Evaluation:** None of the reviewed studies performed a direct comparison between on-policy and off-policy algorithms under the same conditions. This makes it difficult to clearly understand which approach is more suitable for insulin control. In particular, the comparison between PPO and TD3 under stochastic environments has not been properly explored.

The reviewed studies also provide several important insights that guide the design of the present work. Reinforcement learning has consistently shown strong potential for closed-loop insulin control, especially when compared to traditional rule-based or

supervised methods. The ability of RL to continuously adapt based on feedback makes it well-suited for dynamic glucose management.

The UVA/Padova simulator, accessed through the simglucose library, has become a standard platform in this field. It provides a safe and physiologically validated environment for testing RL-based approaches, which is why it is widely used across multiple studies.

Another key observation is that entropy plays an important role in RL training, but it is often not given enough attention. Most existing works use fixed entropy values, which limits performance. This creates a clear gap that is addressed in this project through the adaptive entropy decay mechanism in ER-PPO.

Additionally, the lack of a controlled comparison between PPO and TD3 in previous work leaves an important question unanswered regarding their relative performance in realistic settings. This gap is directly addressed in Module 3 of this project.

Overall, the existing literature shows that while significant progress has been made in applying reinforcement learning to insulin control, several key limitations still remain. Issues related to fixed exploration strategies, lack of generalization, incomplete state representation, and absence of proper comparative studies indicate that current systems are not yet fully optimized for real-world deployment.

The proposed ER-PPO framework is designed to address these gaps by introducing adaptive entropy regulation, using a richer state representation, and evaluating multiple RL algorithms under identical stochastic conditions. By doing so, it aims to provide a more stable, safe, and generalizable solution for closed-loop insulin delivery in Type-1 Diabetes management.

2.3 Software Requirement Specification

The Software Requirement Specification (SRS) defines the functional and non-functional requirements of the proposed Adaptive Insulin Dosage Prediction system. The system spans three development phases and integrates supervised deep learning, RL-based closed-loop control, and the entropy-adaptive ER-PPO algorithm [26].

This document basically gives a clear idea of what the system should do, how it should behave, and what constraints it must follow. It helps in keeping the

development structured and ensures that all modules are aligned with the project objectives.

2.3.1 System Overview

The proposed system consists of three integrated modules corresponding to the three phases of the project.

The first module (Module 1) focuses on data preprocessing and supervised learning using the AIM-94 dataset. It prepares the data and trains prediction models like LSTM, BiLSTM, and TFT.

The second module (Module 2) introduces reinforcement learning by implementing a PPO agent inside the simglucose environment. This allows the system to move from prediction to actual decision-making.

The third module (Module 3) performs a comparative evaluation of PPO, TD3, and ER-PPO under stochastic conditions. Among these, ER-PPO is the main contribution and is treated as the final system output.

Overall, the system gradually evolves from simple prediction to a more realistic and adaptive control system.

2.3.2 Functional Requirements

The system must satisfy the following functional requirements across all modules:

- **Multi-Phase Data Pipeline:** The system should support data loading from both real datasets (AIM-94) and simulated data from the UVA/Padova simulator. It should handle preprocessing like normalization and sequence creation properly.
- **Supervised Prediction Module:** It should implement LSTM, BiLSTM, and TFT models that can process 24-hour input sequences and generate accurate next-hour insulin predictions.
- **RL Environment Interface:** The system must be able to interact with the simglucose environment using OpenAI Gym interface. It should support multiple episodes and allow different meal patterns to be tested.
- **Multi-Agent Training Framework:** PPO, TD3, and ER-PPO agents should be trained separately but under the same conditions. This ensures fair comparison between algorithms.

- **Adaptive Entropy Control:** The ER-PPO model must implement entropy decay from 0.05 to 0.001. This is important because it controls exploration and stability during training.
- **Performance Metric Computation:** The system should calculate important metrics like Time-in-Range, hypoglycemia rate, hyperglycemia rate, mean glucose, and total insulin delivered.
- **Comparative Reporting:** It should generate tables and graphs to compare performance of different models, so results can be easily understood.

2.3.3 Non-Functional Requirements

Apart from functionality, the system must also meet some quality requirements:

- **Safety:** Insulin output must always stay within a safe range. Even during training, unsafe values should be restricted.
- **Reproducibility:** Experiments should be repeatable. For this, fixed random seeds are used so results remain consistent.
- **Modularity:** Different components like reward function, model, and simulator should be independent. So if one part is changed, the whole system need not be rebuilt.
- **Computational Efficiency:** Training should be reasonably fast. Using GPU helps in reducing training time, especially for RL models.
- **Interpretability:** Results should be easy to understand. Graphs, tables, and plots should clearly show system performance.
- **Privacy and Compliance:** All patient data must be anonymised. No personal information should be used in training or testing.

2.3.4 Development Environment

The system is developed using the following tools:

- Python 3.9 is used as the main programming language because of its flexibility and wide support.
- PyTorch 2.0 is used for implementing deep learning models and RL networks.
- PyTorch Forecasting 1.0 is specifically used for the TFT model.
- Stable-Baselines3 v2.1 provides standard implementations of PPO and TD3.
- simglucose v0.2.1 is used as the simulation environment for RL training.

- NumPy, Pandas, and Scikit-learn are used for data processing and evaluation.

These tools together make the system easier to build and experiment with.

2.3.5 Hardware Requirements

- Processor: Intel Core i5 or higher (i7/i9 preferred for faster training)
- RAM: Minimum 8 GB (16 GB is better for smoother performance)
- Storage: At least 10 GB free space
- GPU: NVIDIA GPU with CUDA support is recommended for faster model training

Without GPU, training is still possible but may take more time.

2.3.6 Data Requirements

The system uses two types of data:

- **AIM-94 Dataset:** Contains real patient data with glucose readings, insulin records, and meal details. It should be properly structured and time-indexed.
- **UVA/Padova Simulator:** Generates synthetic but realistic glucose-insulin data for 30 virtual patients. Meal disturbances are randomly generated between 30–80 g at intervals of 30–120 minutes.
- **Privacy and Compliance:** All patient data must be anonymised and handled carefully to avoid misuse.

2.3.7 System Constraints

There are some limitations in the current system:

- The RL models are trained only in simulation, not on real patients
- simglucose does not include sensor noise or device failures
- Training ER-PPO requires GPU for practical execution time
- AIM-94 dataset usage is restricted by data policies

These constraints should be considered when interpreting results.

2.3.8 Assumptions

The system is built based on some assumptions:

- The simulator behaves similar to real human glucose-insulin response
- Meal events are simplified but still realistic enough
- Training checkpoints are saved and reused when needed
- Required hardware resources are available

These assumptions help in simplifying the system design, although in real-world deployment additional factors may need to be considered.

CHAPTER-3

Proposed Method

3.1 Design Methodology

The proposed system is a three-module framework that is structured in such a way that each module builds upon the development of an intelligent insulin dosing system. The methodology advances to supervised learning-based prediction, reinforcement learning-based control, and lastly to an optimized adaptive control strategy. This step-by-step methodology guarantees the systematic growth, performance analysis, and optimization of the model.

Module 1 provides a baseline based on deep learning to forecast insulin dosage based on past patient records. Module 2 presents a closed-loop control system with reinforcement learning in a simulated physiological setting. Module 3 builds on this framework by comparing various reinforcement learning algorithms and suggesting an improved version of Proximal Policy Optimization (PPO) with adaptive entropy regulation to achieve better stability and safety.

3.1.1 Module 1 Methodology

Module 1 aims to forecast the insulin dosage requirement of Type-1 Diabetes patients in the next-hour based on multivariate time-series data. This module is used in an open-loop environment, where forecasts are made only using historical data without real-time feedback.

This is done using the AIM-94 dataset. It is a longitudinal record of 70 patients, which contains continuous glucose, insulin administration, meal intake, and other physiological variables, which are measured at regular intervals. This data set gives a detailed account of the glucose-insulin interactions with time.

A sliding window method is used to capture temporal dependencies. The input samples are 24-hour histories of observations, which allow the model to learn both short-term fluctuations and longer-term physiological trends. The prediction target is defined as the insulin dosage at the next time step (t+1), formulating the task as a regression problem.

Three deep learning models are applied and tested:

Long Short-Term Memory (LSTM), which is useful in sequential dependencies of time-series data.

- Bidirectional LSTM (BiLSTM), which improves the context capture by operating sequences in both forward and backward directions.
- Temporal Fusion Transformer (TFT), a model that uses attention to learn complex temporal relationships and feature significance.

All models are trained with the same preprocessing conditions, such as min-max normalization and chronological division of data into training, validation, and testing sets to maintain the temporal order.

Mean Absolute Error (MAE) and Root Mean Square Error (RMSE) are used to assess model performance and determine the accuracy of the prediction and the magnitude of the error.

Despite offering useful information on insulin prediction, this module lacks the feedback of the surrounding environment, which restricts its capacity to adjust to the real-time changes in glucose.

3.1.2 Module 2 Methodology

Module 2 builds upon the prediction system to real-time decision-making by adding a closed-loop control system with reinforcement learning.

The simglucose library is used to build the simulation environment, and it is the implementation of the UVA/Padova Type-1 Diabetes simulator that is accepted by the FDA. This environment offers a secure and clinically validated training and evaluation platform of control policies.

One virtual patient (adult#001) is employed to provide consistency between experiments. The reinforcement learning agent engages with the environment in episodes of 1000 timesteps, where each timestep is 3 minutes.

The agent monitors the present physiological condition, such as glucose levels and other pertinent historical data, and provides a continuous insulin dosage response. A reward mechanism is created to punish the deviation of the target glucose range

(70180 mg/dL), and the punishment is more severe in the case of hypoglycemic conditions because of their clinical risk.

This module uses the Proximal Policy Optimization (PPO) algorithm. PPO is chosen due to its stability and effectiveness in continuous control problems, which is obtained by its clipped surrogate objective that avoids large policy updates.

The PPO-based system has a Time-in-Range (TIR) of 86.55% when the fixed meal scenario is controlled, which means that the glucose is well regulated in a stable environment.

Nonetheless, the fact that a constant entropy coefficient is used during training restricts the flexibility of the policy. The lack of a system to decrease exploration in later training phases may influence the consistency and reliability of the learned dosing strategy.

3.1.3 Module 3 Methodology

Module 3 is aimed at enhancing the strength and clinical usability of the system by comparing and improving the algorithms.

Three reinforcement learning algorithms are tested:

- Proximal Policy Optimization (PPO)
- Twin Delayed Deep Deterministic Policy Gradient (TD3)

The proposed method is called Entropy-Regulated Proximal Policy Optimization (ER-PPO).

The environment is altered to add stochastic meal disturbances to simulate realistic conditions. The carbohydrate content of meals ranges between 30 and 80 grams and the timing of meals is randomized between 30 and 120 minutes. This brings in variability and questions the flexibility of the control policies.

The suggested ER-PPO algorithm is a modification of the conventional PPO algorithm, which includes a linearly decreasing entropy coefficient. The entropy coefficient is set to a larger value (0.05) to promote exploration in the initial stages of training and is decreased to a smaller value (0.001) as training advances. This method facilitates a seamless process of exploration to exploitation whereby the agent initially learns effective strategies and then optimizes them into consistent policies.

ER-PPO model has a Time-in-Range (TIR) of 81.60% and a hypoglycemia rate of 1.50%. These outcomes are within and above the clinically recommended threshold of 75% TIR, and low risk of hypoglycemia.

On the whole, this module shows that adaptive entropy regulation can enhance the stability of policies and increase the safety of insulin delivery systems based on reinforcement learning.

3.2 System Architecture

The system architecture is designed as a closed-loop reinforcement learning control framework for automated insulin delivery. It integrates three core components:

1. the physiological simulation environment,
2. the reinforcement learning (RL) agent, and
3. the policy optimization mechanism specific to each algorithm.

Figures **3.1, 3.2, and 3.3** illustrate the system architecture for PPO, ER-PPO, and TD3 respectively. While the overall control pipeline remains consistent across all models, the internal learning mechanisms differ based on the algorithm used.

3.2.1 General Closed-Loop Control Framework

Across all architectures, the system operates in a continuous feedback loop:

- The UVA/Padova simulator (via simglucose) models the patient's glucose-insulin dynamics
- The simulator provides the current state representation, including glucose levels and relevant history
- The RL agent processes this state and outputs an insulin dosage (action)
- The action is applied to the simulator, resulting in new glucose levels
- A reward signal is computed based on deviation from the target glucose range
- The state and reward are fed back to the agent for learning

This loop continues over multiple timesteps, enabling the agent to learn an optimal insulin dosing policy.

3.2.2 PPO-Based Architecture

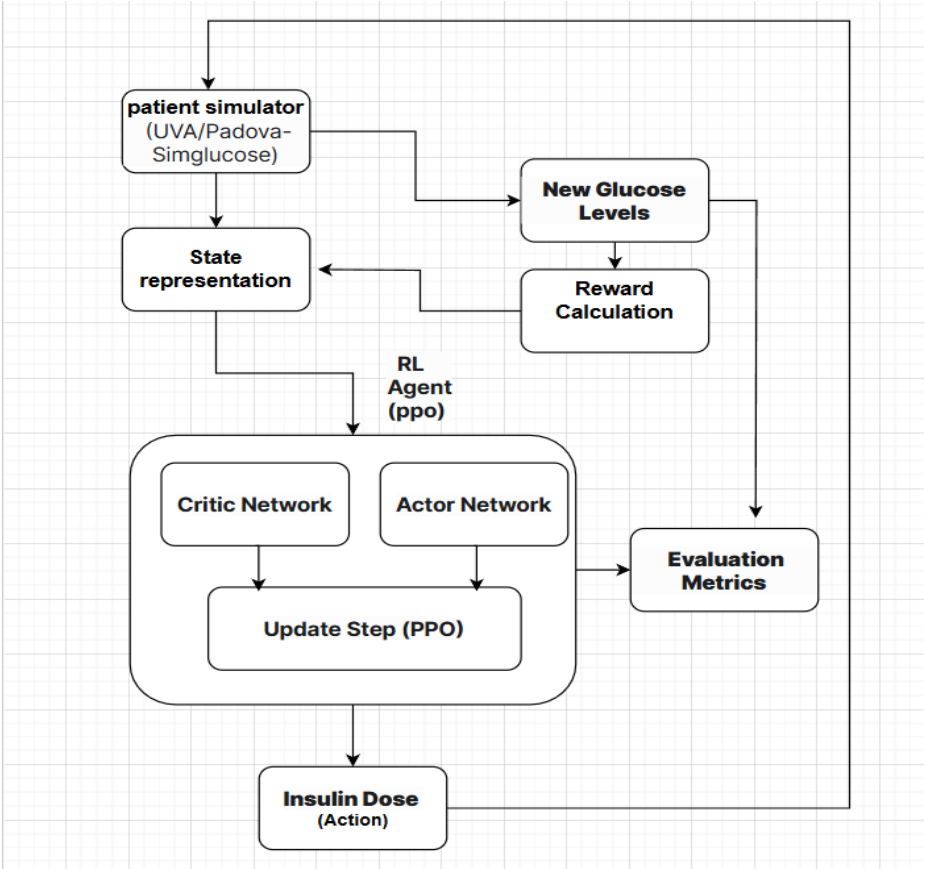

Figure 3.1: Architecture for Phase 2 (PPO)

Figure 3.1 presents the architecture of the PPO-based closed-loop control system.

In this setup:

- The RL agent consists of two neural networks:
  - Actor Network: Generates a stochastic insulin dose based on the current state
  - Critic Network: Estimates the value function to guide policy improvement
- The PPO algorithm updates the policy using a clipped surrogate objective, ensuring stable learning by limiting large policy updates.
- The generated insulin dose is applied to the simulator, producing updated glucose levels and reward signals.
- These signals are used to iteratively refine both actor and critic networks.

This architecture provides stable learning and achieves strong performance under controlled conditions. However, the exploration behavior is governed by a fixed entropy coefficient, which does not adapt during training.

3.2.3 ER-PPO Architecture

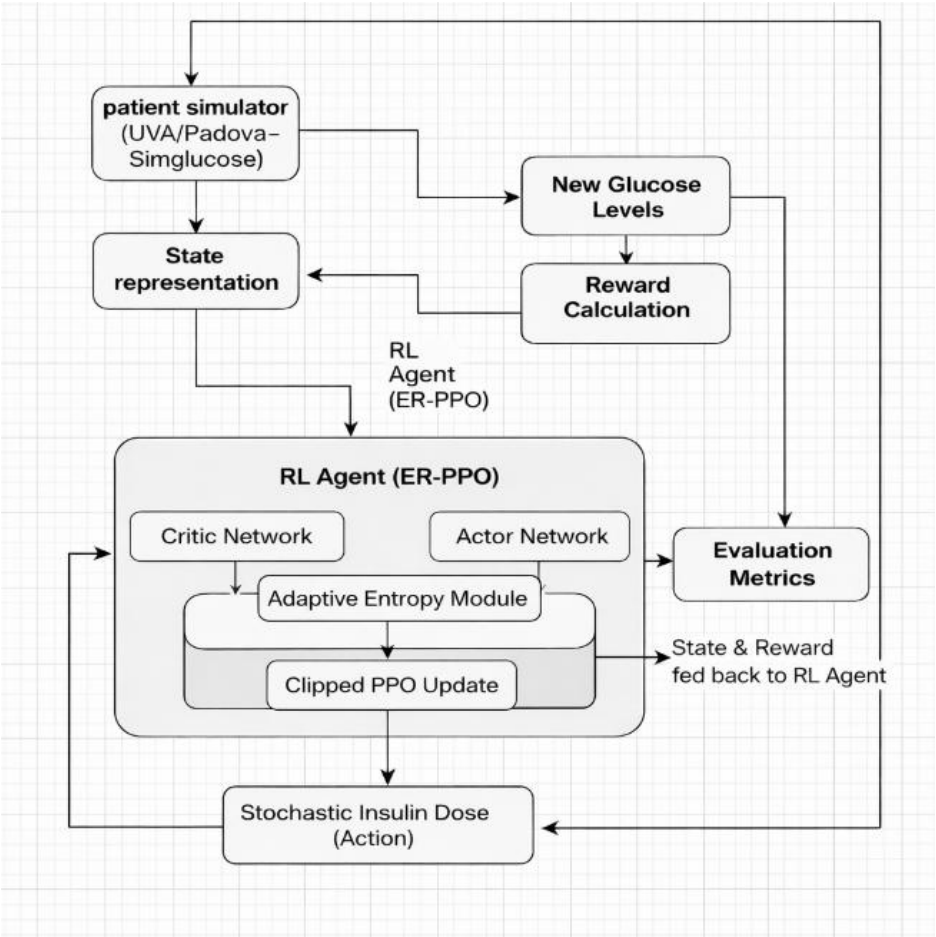

Figure 3.2: Closed-Loop RL Control Architecture for Phase 3 (ER-PPO)

Figure 3.2 illustrates the proposed Entropy-Regulated PPO (ER-PPO) architecture, which extends the standard PPO framework.

The overall structure remains similar to PPO, with actor and critic networks interacting with the environment. However, a key addition is the Adaptive Entropy Module, which dynamically regulates the exploration behavior of the policy.

Key components include:

- Actor and Critic Networks: Same as PPO, responsible for action generation and value estimation

- Adaptive Entropy Module:
    - Controls the entropy coefficient during training
    - Starts with a higher value to encourage exploration
    - Gradually decreases to promote stable policy convergence
- Clipped PPO Update: Incorporates the entropy term into the loss function, balancing exploration and exploitation

Unlike standard PPO, the insulin dosing action here is stochastic but progressively stabilizes as training advances due to entropy decay.

This modification improves:

- Policy consistency in later stages
- Safety of insulin dosing decisions
- Overall robustness under variable conditions

3.2.4 TD3 Architecture

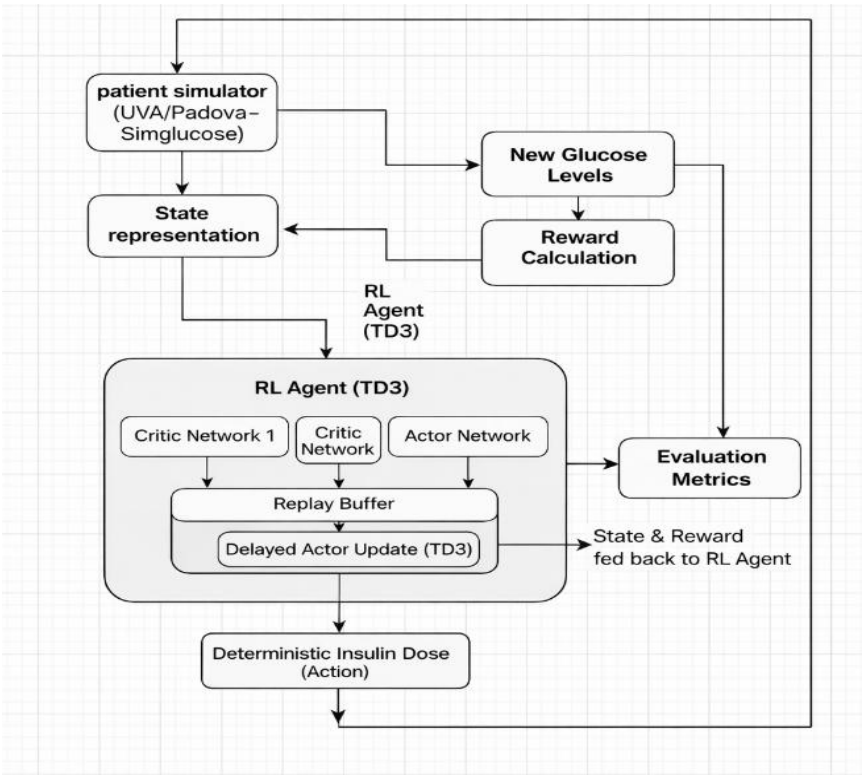

Figure 3.3: Closed-Loop RL Control Architecture for Phase 3 (TD3)

Figure 3.3 presents the architecture of the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm.

This architecture differs fundamentally from PPO-based methods:

- The policy is deterministic, producing a fixed insulin dose for a given state
- The RL agent consists of:
    - Actor Network: Outputs deterministic insulin action
    - Two Critic Networks: Reduce overestimation bias by taking the minimum of value estimates
- A Replay Buffer stores past experiences, enabling off-policy learning
- Delayed Actor Updates: The actor network is updated less frequently than the critics to stabilize training

The action generated is applied to the simulator, and the resulting state transitions are stored in the replay buffer for training.

While TD3 improves stability in value estimation, its deterministic nature limits exploration capability, which can reduce adaptability under highly variable meal conditions.

3.2.5 Key Architectural Differences

Although all three architectures share a common closed-loop structure, their differences lie in how learning and exploration are handled:

- **PPO:** Stochastic policy with fixed exploration
- **ER-PPO:** Stochastic policy with adaptive exploration (entropy decay)
- **TD3:** Deterministic policy with replay-based learning

Among these, ER-PPO introduces an additional control mechanism that enables a balance between exploration and stability, making it more suitable for safety-critical applications such as insulin dosing.

3.3 Description of Algorithms

This section presents the pseudocode and mathematical formulations for all algorithms employed across the three project phases. Each algorithm is specified with its required inputs, guaranteed outputs, and step-by-step computational procedure.

3.3.1 Module 1 — Deep Sequential Models for Insulin Dosage Prediction

Algorithm 1: Temporal Sliding Window Data Preprocessing

```
Require: Raw dataset D (timestamps, glucose, insulin, meals, covariates)
Ensure:  Input tensor X, target vector y
Merge all patient records into a single time-indexed dataframe DF.
Resample DF to 1-hour intervals using mean aggregation and linear interpolation.
Handle remaining missing values using forward-fill then backward-fill.
Normalize all numerical columns to [0, 1] via min-max scaling.
Initialise empty lists X_seq and y_seq.
For each patient p in DF do
    Extract time-ordered sub-dataframe DF_p for patient p.
    For t = 24 to length(DF_p) - 1 do
        window ← DF_p[t-24 : t-1]   {24-hour input window}
        target ← DF_p.insulin[t]    {next-hour insulin dose}
        Append window to X_seq; append target to y_seq.
    End for
End for
Convert X_seq and y_seq into tensors X and y.
Split X, y chronologically (70/15/15) into train/validation/test sets.
Return X_train, y_train, X_val, y_val, X_test, y_test.
```

Algorithm 2: LSTM / BiLSTM Training for Insulin Dosage Prediction

Require: X_train, y_train, X_val, y_val; model type M in {LSTM, BiLSTM}

Ensure: Trained model weights W*, validation MAE and RMSE

1. Instantiate sequential model M with two stacked recurrent layers.
2. If M = BiLSTM: configure bidirectional processing (forward and reverse).
3. Add dropout layer (rate = 0.2) after each recurrent layer.
4. Append dense output layer with a single linear unit for regression.
5. Compile M with Adam optimiser (lr = 0.001) and Mean Squared Error loss.
6. Initialise early stopping with patience = 10 on validation loss.
7. For each epoch e = 1 to E_max do
8. For each mini-batch (x_b, y_b) in training set do
9. y_hat ← M(x_b)
10. L ← MSE(y_hat, y_b)
11. Backpropagate gradients; update W via Adam.
12. End for
13. Compute validation loss; if early stopping criterion met: break.
14. End for
15. Evaluate M on X_test; compute MAE and RMSE.
16. Return W*, MAE, RMSE.

The MSE loss function used during training is defined as:

$$L_{MSE} = \frac{1}{n} \sum_i (\tilde{y}_i - y_i)^2$$

Algorithm 3: Temporal Fusion Transformer (TFT) Training

Require: X_train, y_train, X_val, y_val; sequence length T = 24

Ensure: Trained TFT weights W*, MAE and RMSE

1. Initialise TFT with Variable Selection Networks, Gated Residual Networks, LSTM encoder-decoder, multi-head temporal self-attention, and output projection.
2. Embed static patient covariates and feed to context vectors for GRNs.
3. For each training epoch e = 1 to E_max do
4. For each mini-batch (x_b, y_b) do
5. Pass x_b through variable selection to identify informative inputs.
6. Encode temporal sequence through LSTM encoder-decoder.
7. Apply multi-head attention over encoder outputs.
8. Project attention output to predicted dosage y_hat_b.
9. Compute quantile loss; backpropagate; update W.
10. End for
11. Validate on X_val; apply early stopping if no improvement.
12. End for
13. Evaluate on X_test; compute MAE and RMSE.
14. Return W*, MAE, RMSE.

The multi-head self-attention mechanism employed in TFT computes:

$$Attention(Q, K, V) = \text{softmax}\frac{QK^T}{\sqrt{d_k}} \times V$$

3.3.2 Module 2 — Proximal Policy Optimization (PPO) Baseline

PPO constrains policy updates using a clipped surrogate objective, preventing destructively large gradient steps while maintaining computational efficiency relative to second-order methods.

Algorithm 4: PPO for Closed-Loop Insulin Control (Phase 2 Baseline)

```
Require: Environment E, actor pi_theta, critic V_phi,
        clip ratio eps = 0.2, discount gamma = 0.99, entropy coeff alpha = 0.01
Ensure:  Trained policy pi_theta*
Initialise actor parameters theta and critic parameters phi randomly.
For each training episode i = 1 to N do
    Reset E; observe initial state s_0.
    Collect trajectory tau = {s_t, a_t, r_t} for T = 1000 timesteps:
        Sample a_t ~ pi_theta(.|s_t); apply to E; receive r_t, s_{t+1}.
    Compute discounted returns R_t and GAE advantages A_hat_t.
    For each PPO update epoch k = 1 to K do
        For each mini-batch (s, a, A_hat, R) from tau do
            rho = pi_theta(a|s) / pi_theta_old(a|s)
            L_CLIP = min(rho*A_hat, clip(rho, 1-eps, 1+eps)*A_hat)
            L_V    = (V_phi(s) - R)^2
            L_H    = -alpha * H(pi_theta(.|s))
            L_total = -L_CLIP + c*L_V + L_H
            Update theta and phi by gradient descent on L_total.
        End for
    End for
End for
```

18. Return pi_theta*.

The PPO clipped surrogate objective is defined as:

$$L_{CLIP}(\theta) = \mathbb{E}_t\left(min\left(\rho_t(\theta) \cdot \hat{A}_t, clip(\rho_t(\theta), 1-\varepsilon, 1+\varepsilon) \cdot \hat{A}_t\right)\right)$$

$$where\ \rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{old}(a_t \mid s_t)}$$

3.3.3 Module 3 — PPO, TD3, and ER-PPO (Comparative Algorithms)

Phase 3 evaluates three algorithms under identical stochastic conditions. Standard PPO from Phase 2 is retained as the on-policy reference. TD3 is evaluated as an off-policy deterministic actor-critic method. ER-PPO is the primary contribution of this work, extending PPO with a linearly decaying entropy schedule.

Algorithm 5: Twin Delayed Deep Deterministic Policy Gradient (TD3)

Require: Environment E, actor mu_theta, twin critics Q_phi1 and Q_phi2,

replay buffer B, policy noise sigma = 0.2, update delay d = 2

Ensure: Trained deterministic policy mu_theta*

1. Initialise all networks, target networks, and replay buffer B.
2. For each timestep t = 1 to T_total do
3. a_t = mu_theta(s_t) + eps, eps ~ N(0, sigma_explore)
4. Execute a_t; observe r_t and s_{t+1}; store in B.
5. If |B| >= batch size then
6. Sample mini-batch from B.
7. Smoothed target: a_tilde = mu_theta_target(s') + clip(N(0,sigma), -c, c)
8. TD target: y = r + gamma * min(Q_phi1_target(s',a_tilde), Q_phi2_target(s',a_tilde))
9. Update critics by minimising (Q_phi_i(s,a) - y)^2 for i = 1, 2.

10. If t mod d = 0 then {delayed policy update}

11. Update actor: maximise Q_phi1(s, mu_theta(s)).

12. Soft-update target networks: theta' ← tau*theta + (1-tau)*theta'

13. End if

14. End if

15. End for

16. Return mu_theta*.

The TD3 target value uses twin critics to mitigate Q-value overestimation:

$$y = r + \gamma \cdot min_{i=1,2} Q_{\varphi,i}^{target}(s', \mu_\theta(s') + \varepsilon), \qquad \varepsilon \sim clip(\mathcal{N}(0,\sigma), -c, c)$$

Algorithm 6: ER-PPO — Entropy-Regulated Proximal Policy Optimization (Novel Contribution)

Require: Environment E, actor pi_theta, critic V_phi, total episodes N,

alpha_start = 0.05, alpha_end = 0.001, clip eps = 0.2, gamma = 0.99

Ensure: Trained adaptive policy pi_theta*

1. Initialise theta and phi randomly.

2. For each training episode i = 1 to N do

3. Compute epoch-adaptive entropy coefficient:

alpha_i = alpha_start - (alpha_start - alpha_end) * (i / N)

4. Reset E; observe s_0 = [BG, DeltaBG, IOB, COB].

5. Collect trajectory tau = {s_t, a_t, r_t} for T = 1000 timesteps:

6. Sample a_t ~ pi_theta(.|s_t), a_t in [0, 0.2] U.

7. Apply a_t; receive r_t and s_{t+1}.

8. Compute reward:

```
        if 70 ≤ BG ≤ 180:        r_t = +1.0
        elif mild deviation:      r_t = -0.5
        elif dangerous excursion:  r_t = -2.0 - lambda*|Delta_a|
    Compute returns R_t and GAE advantages A_hat_t.
    For each PPO update epoch k = 1 to K do
        For each mini-batch from tau do
            rho  ← pi_theta(a|s) / pi_theta_old(a|s)
            L_CLIP ← min(rho*A_hat, clip(rho, 1-eps, 1+eps)*A_hat)
            L_V    ← (V_phi(s) - R)^2
            L_H    ← -alpha_i * H(pi_theta(.|s))   {adaptive entropy}
            L_total ← -L_CLIP + c*L_V + L_H
            Update theta and phi by gradient descent on L_total.
        End for
    End for
End for
Return pi_theta*.
```

The key innovation of ER-PPO is the epoch-adaptive entropy coefficient. The total training objective is:

$$L_{ER-PPO}(\theta) = -L_{CLIP}(\theta) + c_1 \cdot L_V(\varphi) - \alpha_i \cdot H(\pi_\theta)$$

$$where\ \alpha_i = \alpha_{start} - (\alpha_{start} - \alpha_{end}) \cdot \frac{i}{N}$$

3.4 RL Environment Design, Dataset and Tools

3.4.1 State Space

The state space defines what information is available to the RL agent at each timestep, and it plays a very important role in how well the model can learn.

In the ER-PPO model, the state is represented using four features: current blood glucose level (BG), rate of change of blood glucose (ΔBG), Insulin on Board (IOB), and Carbohydrates on Board (COB). This combination gives both current and short-term future information.

The inclusion of ΔBG is especially useful because it tells whether glucose is increasing or decreasing. Without this, the model only sees the current value and may react too late.

In contrast, the TD3 agent uses only three features: BG, IOB, and COB. Since ΔBG is not included, the model does not have information about glucose trend direction. Because of this, it often reacts after glucose has already changed significantly, which can lead to unstable control.

So even though the difference looks small (just one feature), it actually affects performance quite a lot.

Table 3.1: State Space Comparison — ER-PPO vs TD3

Feature	ER-PPO State	TD3 State	Description
BG	Yes	Yes	Blood glucose level (mg/dL)
Delta BG	Yes	No	Rate of change of blood glucose
IOB	Yes	Yes	Insulin on Board (U)
COB	Yes	Yes	Carbohydrates on Board (g)
Dimension	4	3	State vector dimensionality

3.4.2 Action Space

The action space represents the insulin dose selected by the agent at each timestep.

For ER-PPO and PPO, the action space is continuous and limited between [0, 0.2] units per timestep. This range is chosen carefully to represent a safe basal insulin delivery, avoiding sudden large doses.

TD3, on the other hand, uses a wider range of [0, 1.6] units. While this allows the model to respond strongly to high glucose levels, it also increases the risk of overdosing.

In practice, this wider range becomes a problem. The model tends to use large insulin values frequently, which leads to hypoglycemia. So even though a larger range seems flexible, it is not always safe in medical applications.

3.4.3 Reward Function Design

The reward function is what guides the learning process of the RL agent. If the reward is not designed properly, the model may learn wrong behavior.

In ER-PPO, the reward is divided into three zones based on glucose levels, along with a smoothness penalty to reduce sudden changes in insulin.

- When glucose is in the safe range (70–180 mg/dL), the model gets a positive reward (+1.0)
- For mild deviations, a small penalty is given
- For dangerous conditions, a strong negative reward is applied

Additionally, a smoothness term is included so that the model avoids giving very sharp or irregular insulin doses. This helps in maintaining stability.

TD3 uses a Gaussian-based reward centered around normal glucose values. However, this type of reward does not strongly penalize hypoglycemia, which is why the model ends up producing unsafe insulin actions.

So overall, ER-PPO reward design is more aligned with clinical safety.

Table 3.2: ER-PPO Reward Function Design

Glycemic Condition	BG Range (mg/dL)	ER-PPO Reward	Description
In-Range (Target)	70 to 180	+1.0	Full positive reward for safe glucose level
Mild Deviation	54-70 or 180-250	-0.5	Moderate penalty; policy encouraged to recover
Dangerous Excursion	Below 54 or above 250	-2.0 minus smoothness penalty	Severe penalty plus action smoothness term to discourage erratic dosing

3.4.4 AIM-94 Dataset

The AIM-94 dataset is used for training the supervised learning models in Module 1.

It contains data from 70 Type-1 diabetic patients, including glucose readings, insulin doses, meal information, and other physiological parameters. The data is recorded over time, so it forms a multivariate time-series dataset.

For this project, the data is resampled into hourly intervals, which makes it easier to train sequence models like LSTM and TFT. In total, around 87,000 data points are used.

This dataset is quite useful because it represents real patient behavior, although it is still limited compared to real-time continuous monitoring systems.

3.4.5 UVA/Padova T1D Simulator (simglucose)

For reinforcement learning, a real patient cannot be used for training, so a simulation environment is required.

The simglucose library provides an implementation of the UVA/Padova simulator, which is widely accepted and even approved for research use. It models glucose-insulin dynamics using mathematical equations.

In this project, the adult#001 virtual patient is used for training and testing. Each episode consists of 1000 timesteps, with each step representing 3 minutes.

To make the environment more realistic, random meal events are introduced. The carbohydrate intake varies between 30–80 grams, and meal timings are also random between 30–120 minutes.

This randomness is important because it prevents the model from memorizing patterns and forces it to generalize.

3.4.6 Tools and Frameworks

The system is implemented using a combination of different tools and libraries.

Python (version 3.9) is used as the main programming language. PyTorch is used for building deep learning models like LSTM, BiLSTM, TFT, and also for RL policy networks.

PyTorch Forecasting is used specifically for implementing the TFT model. Stable-Baselines3 is used for reinforcement learning algorithms like PPO and TD3.

The simglucose library provides the simulation environment, while pandas and NumPy are used for data processing. Matplotlib and seaborn are used for visualization of results.

Jupyter Notebook is used during development for testing and experimentation, and Git/GitHub is used for version control.

All these tools together help in building, training, and evaluating the complete system.

Table 3.3: Development Environment and Tools

Component	Tool / Library	Version	Purpose
Programming Language	Python	3.9	Core implementation language
Deep Learning	PyTorch	2.0	LSTM, BiLSTM, TFT, and RL policy networks

TFT Architecture	PyTorch Forecasting	1.0	Temporal Fusion Transformer implementation
RL Framework	Stable-Baselines3	2.1	Standardized PPO and TD3 base implementations
Simulation Environment	simglucose	0.2.1	UVA/Padova T1D simulator via OpenAI Gym
Data Manipulation	pandas / NumPy	2.0 / 1.25	Data preprocessing and numerical computation
Visualization	matplotlib / seaborn	3.7 / 0.12	Result plotting and figure generation
Notebook Environment	Jupyter Notebook	7.0	Interactive experimentation and prototyping
Version Control	Git / GitHub	---	Code versioning and repository management

CHAPTER-4

Results and Observations

4.1 Stepwise Description of Results

4.1.1 Module 1 Results

Module 1 evaluated three deep learning architectures — LSTM, BiLSTM, and Temporal Fusion Transformer (TFT) — for supervised next-hour insulin dosage prediction on the AIM-94 dataset comprising 70 Type-1 diabetic patients. Performance is measured using Mean Absolute Error (MAE) and Root Mean Square Error (RMSE).

Table 4.1: Module 1 — Model Performance on AIM-94 Test Set

Model	MAE (U)	RMSE (U)
LSTM	0.6860	1.7758
BiLSTM	0.8034	1.8837
TFT	0.3954 (Best)	1.8634

The TFT achieved the lowest MAE of 0.3954 U, representing a 42.4% improvement over LSTM (0.6860 U) and 50.8% over BiLSTM (0.8034 U). TFT's multi-head temporal self-attention mechanism enables selective focus on the most informative timesteps, unlike recurrent layers, which propagate influence uniformly.

RMSE values across all three models converge in the 1.77–1.88 U range, indicating that peak prediction errors are broadly comparable irrespective of architecture.

BiLSTM underperformed the unidirectional LSTM on both metrics despite its additional backward temporal pass, likely attributable to the causal constraint of clinical insulin prescribing, where future glucose readings are unavailable at decision time.

All Module 1 models share a fundamental structural limitation: they operate in an open-loop setting, without any mechanism to incorporate the patient's glycemic response after a dose is administered.

4.1.2 Module 2 Results

Module 2 introduced closed-loop insulin delivery using Proximal Policy Optimization (PPO) within the simglucose UVA/Padova simulation environment. The agent was trained on the adult#001 virtual patient under a controlled, deterministic meal scenario and evaluated over 1000 timesteps per episode.

Table 4.2: Module 2 — PPO Baseline Under Deterministic Meal Conditions

Metric	PPO — Phase 2 Baseline	Clinical Benchmark
Time-in-Range (70–180 mg/dL)	86.55%	≥ 75%
Meal Scenario	Controlled (fixed)	—
Episode Length (steps)	1,000	—

The Module 2 PPO agent achieved a Time-in-Range of 86.55%, surpassing the internationally recommended 75% TIR clinical benchmark. This result confirms the viability of policy-gradient reinforcement learning for closed-loop insulin control in a physiological simulator.

However, the controlled meal scenario presents an unrealistically favorable training condition, as fixed meal timing allows the policy to exploit predictable disturbance patterns.

When random meal events were introduced in Module 3, the same PPO configuration achieved only 65.36% TIR — a drop of more than 20 percentage points — exposing the brittleness of a policy trained in a static environment.

This degradation, combined with the fixed entropy coefficient that impairs late-stage convergence, directly motivates the adaptive entropy mechanism in ER-PPO.

4.1.3 Module 3 Results

Module 3 constitutes the primary experimental contribution of this work. Three RL algorithms — PPO, TD3, and ER-PPO — were trained and evaluated under identical stochastic conditions: random meal disturbances of 30–80 g carbohydrates at 30–120 minute intervals on the adult#001 virtual patient over 1000 timesteps per episode.

Table 4.3: Module 3 — Comparative Results: PPO vs TD3 vs ER-PPO (Stochastic Meals)

Metric	PPO	TD3	ER-PPO
Time-in-Range (70–180 mg/dL)	65.36%	64.30%	81.60%
Hypoglycemia Rate (<70 mg/dL)	14.84%	35.70%	1.50%
Hyperglycemia Rate (>180 mg/dL)	19.79%	0.00%	16.90%
Mean Blood Glucose (mg/dL)	130.55	97.74	140.25
Glucose Std Dev (mg/dL)	54.03	~28.00	~35.00
Total Insulin Delivered (U)	4.09	14,008	30.67
Policy Type	On-policy	Off-policy	On-policy + Entropy Decay

4.1.3.1 Time-in-Range Analysis

ER-PPO achieves a TIR of 81.60% — the only algorithm in this study to surpass the 75% clinical threshold. Standard PPO and TD3 achieve 65.36% and 64.30%,

respectively, both falling approximately 10 percentage points below the clinical benchmark.

The superior TIR of ER-PPO is directly attributable to its adaptive entropy decay mechanism. A high entropy coefficient ($\alpha = 0.05$) in early training encourages broad exploration of the insulin dosing action space, allowing the agent to discover safe strategies across diverse meal scenarios. As training advances, entropy decays to $\alpha = 0.001$, progressively narrowing the policy toward focused, reliable dosing behavior.

4.1.3.2 Hypoglycemia and Clinical Safety Analysis

ER-PPO records a hypoglycemia rate of 1.50%, well within the clinical target of less than 4% time below 70 mg/dL specified by contemporary T1D management guidelines. Standard PPO records 14.84% — nearly four times the safety threshold.

TD3 produces a catastrophic hypoglycemia rate of 35.70%, meaning nearly one-third of the time is spent in a dangerously low glycemic state.

This is compounded by TD3's total insulin delivery of 14,008 U — approximately 457 times more than ER-PPO (30.67 U). The TD3 reward function, based on a Gaussian centered near 115 mg/dL with insufficient hypoglycemia penalties, combined with its aggressive action range of 0–1.6 U, drives chronic insulin overdosing.

4.1.3.3 Hyperglycemia and Mean Glucose Analysis

TD3 achieves 0.00% hyperglycemia — a superficially favorable result that is entirely explained by chronic insulin overdosing. By suppressing glucose to hypoglycemic levels, TD3 avoids hyperglycemia at the cost of severe safety violations.

ER-PPO records 16.90% hyperglycemia, and PPO records 19.79%, both reflecting the challenge of managing post-meal glucose spikes under stochastic conditions.

The mean blood glucose for ER-PPO is 140.25 mg/dL, within the clinically acceptable range. The glucose standard deviation for ER-PPO (~35 mg/dL) is significantly lower than PPO (54.03 mg/dL), indicating a more stable glycemic profile.

4.1.3.4 Visual Results — PPO Blood Glucose Trajectory

Figure 4.1 and Figure 4.2 present the PPO terminal evaluation output and the glucose-insulin trajectory respectively.

```
--- Evaluation Run Complete ---

--- Quantitative Clinical Evaluation Report ---

Clinical Outcomes:
  - Time in Range (70-180 mg/dL): 65.36 %
  - Time Below Range (<70 mg/dL):  14.84 %
  - Time Above Range (>180 mg/dL): 19.79 %
  - Mean Glucose:                 130.55 mg/dL
  - Glucose Variability (Std Dev):54.03 mg/dL

Agent Behavior:
  - Total Insulin Administered:     4.09 U
```

Figure 4.1: PPO Agent — Terminal Evaluation Output

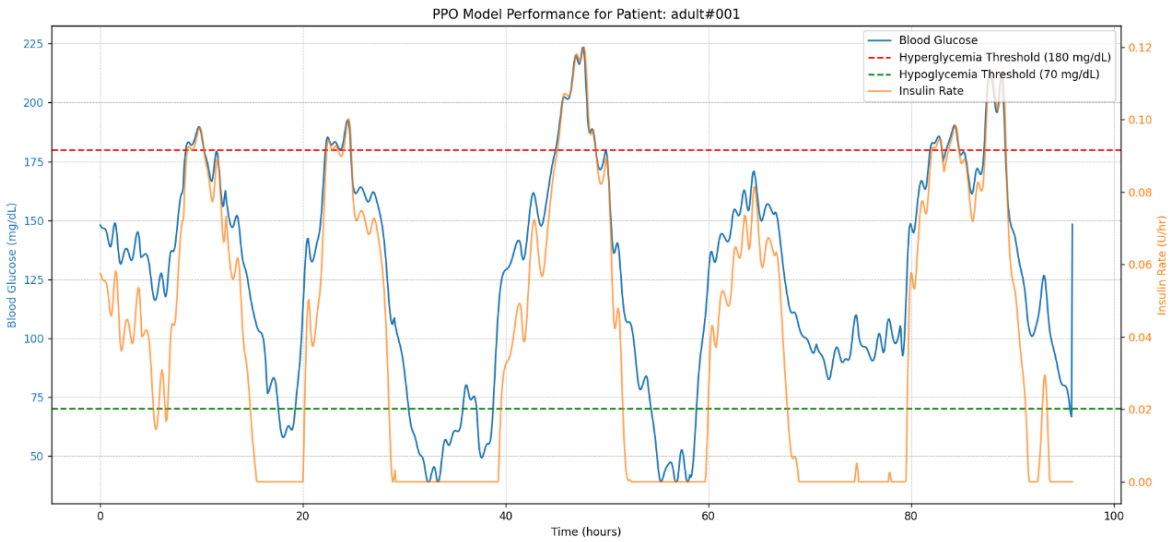

Figure 4.2: PPO Agent — Blood Glucose and Insulin Rate Trajectory (adult#001)

The terminal output confirms a Time-in-Range (TIR) of 65.36%, hypoglycemia rate of 14.84%, and total insulin delivered of 4.09 U. From these values itself, it can be

seen that although PPO is somewhat working, it is not maintaining safe glucose levels consistently.

Looking at the glucose trajectory, there are clear post-meal spikes where glucose rises above 180 mg/dL, followed by sharp drops going below 70 mg/dL. This shows that the model is reacting after glucose increases, rather than preventing it earlier.

Another thing noticeable is that insulin delivery is somewhat irregular. Sometimes it gives higher doses suddenly, and then stops, which leads to these oscillations. So basically, the model has learned a reactive behaviour — it corrects errors after they happen, but it is not good at maintaining stable glucose levels continuously.

Also, the high hypoglycemia percentage (14.84%) is a serious issue clinically. It means the model is over-correcting in many situations. So even though PPO works in controlled conditions, here it struggles when the environment becomes unpredictable.

4.1.3.5 Visual Results — TD3 Blood Glucose Trajectory

Figure 4.3 and Figure 4.4 present the TD3 terminal results and corresponding glucose trajectory.

```
📊 --- TD3 RESULTS ---
Model:          TD3_FINAL_STABLE_20260327_103858
TIR:            64.30%
Hypo:           35.70%
Hyper:          0.00%
Mean Glucose:   97.74
Min Glucose:    39.00
Max Glucose:    154.42
Total Insulin:  14008.6182
```

Figure 4.3: TD3 Agent — Terminal Evaluation Output

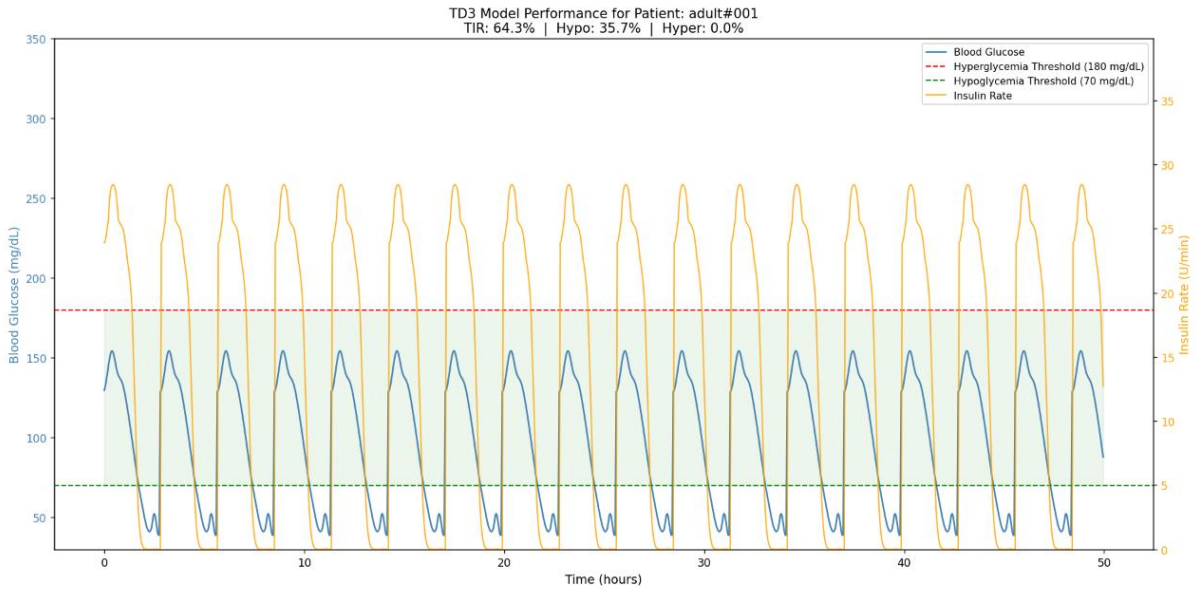

Figure 4.4: TD3 Agent — Blood Glucose and Insulin Rate Trajectory (adult#001)

The terminal output shows TIR of 64.30%, hypoglycemia rate of 35.70%, and total insulin delivered of 14,008 U. These values itself indicate that something is seriously wrong with the policy learned by TD3.

From the graph, a very clear oscillatory pattern can be seen. Every time glucose increases slightly, the model gives a very large insulin dose, which immediately drops glucose to very low levels. This cycle keeps repeating again and again.

This behaviour is not just inefficient, but dangerous. The glucose repeatedly goes below safe levels, and stays there for a considerable amount of time. The hypoglycemia rate of 35.70% means almost one-third of the time the patient is in a risky condition.

Also, the insulin usage is extremely high compared to PPO and ER-PPO. This clearly shows that TD3 is not learning a balanced control strategy. Instead, it is exploiting the reward function in a wrong way — trying to avoid high glucose by aggressively lowering it.

So even though TD3 is a strong algorithm in general RL problems, in this medical setting it fails because there are no strict safety constraints.

4.1.3.6 Visual Results — ER-PPO Blood Glucose Trajectory

Figure 4.5 and Figure 4.6 show the ER-PPO terminal output and glucose-insulin trajectory.

```
📊 --- ER-PPO RESULTS ---
TIR:            81.60%
Hypo:           1.50%
Hyper:          16.90%
Mean Glucose:   140.25
Total Insulin:  30.6736
```

Figure 4.5: ER-PPO Agent — Terminal Evaluation Output

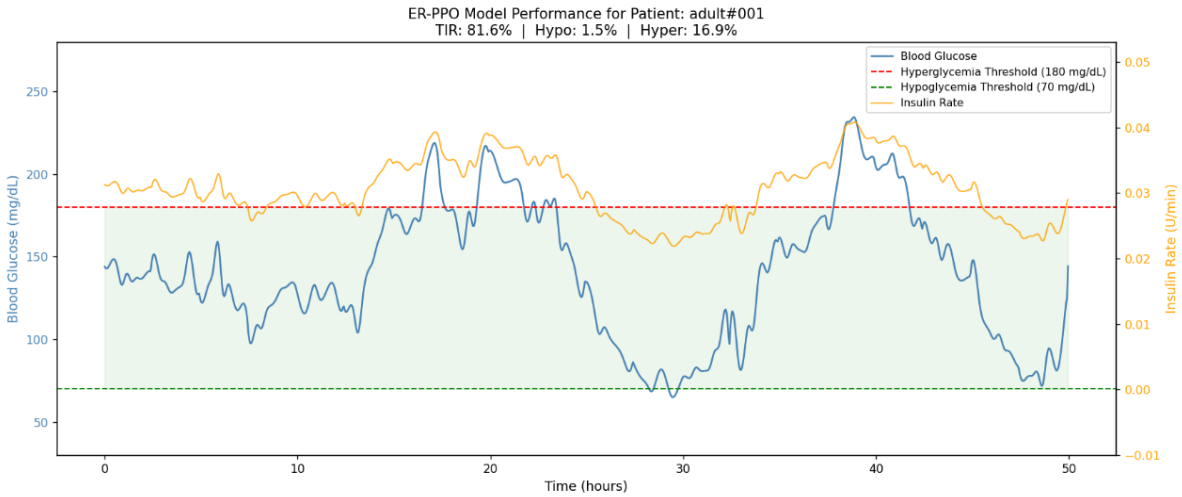

Figure 4.6: ER-PPO Agent — Blood Glucose and Insulin Rate Trajectory (adult#001)

The results confirm a TIR of 81.60%, hypoglycemia of 1.50%, and total insulin delivery of 30.67 U. These values clearly indicate that ER-PPO is performing much better compared to the other two models.

From the graph, the glucose levels remain mostly within the target range (70–180 mg/dL). There are some spikes after meals, which is expected, but they are controlled smoothly without causing sharp drops.

The insulin curve is also much smoother compared to PPO and TD3. Instead of sudden large doses, the model gives gradual adjustments. This helps in avoiding both hyperglycemia and hypoglycemia.

Another important observation is that even when glucose rises above the range, it comes back slowly without overshooting. This means the model is not overreacting.

The two moderate spikes seen in the graph are likely due to larger meal disturbances, but even then, the system recovers without going into dangerous hypoglycemic levels.

Overall, ER-PPO shows a much more stable and balanced behaviour. It is neither too aggressive like TD3 nor too reactive like PPO. It maintains a good trade-off between safety and performance.

4.2 Result Analysis

Table 4.4 situates the performance of each project module within a unified summary, tracing the progression from static prediction through deterministic closed-loop control to adaptive stochastic control.

Table 4.4: Cross-Module Performance Summary (*Controlled meal scenario*)

Module	Method	Key Metric	Limitation Resolved
Module 1	Supervised DL (TFT)	MAE: 0.3954 U	Prediction baseline established
Module 2	PPO (RL)	TIR: 86.55%	Closed-loop feedback introduced
Module 3	ER-PPO (RL)	TIR: 81.60%	Stochastic meals; entropy-regulated policy

Module 1 demonstrated that data-driven deep learning can approximate insulin requirements with sub-unit accuracy from historical patient records; however, the absence of feedback prevents adaptation to real-time glycemic changes. The model predicts well, but it does not “react,” which is a major limitation in real-world glucose control where conditions change continuously.

Module 2 addressed this limitation by introducing reinforcement learning and a feedback loop. The PPO agent was able to dynamically adjust insulin based on current glucose levels, achieving 86.55% TIR under controlled conditions. This clearly shows that RL is more suitable for real-time medical control problems compared to static prediction models.

However, this performance was achieved under fixed meal conditions, which is not realistic. When the same PPO model was exposed to stochastic meal patterns in Module 3, the TIR dropped significantly to 65.36%. This indicates that the model had overfitted to predictable patterns and struggled to generalize.

Module 3 resolves this issue by introducing ER-PPO, which incorporates adaptive entropy regulation. Unlike standard PPO, it allows controlled exploration during early training and gradually stabilizes the policy. This results in better generalization and improved performance under realistic conditions.

Overall, the progression across modules shows a clear evolution:

- From prediction (Module 1)
- To control (Module 2)
- To robust and adaptive control (Module 3)

This progression is important because each module is not isolated, but builds on the limitations of the previous one.

4.3 Observations from the Work

4.3.1 Entropy Regulation Has Direct Clinical Consequences

One of the most important observations from this work is that entropy is not just a training parameter, but it directly affects clinical safety.

When entropy is kept high throughout training, the model continues to behave unpredictably even in later stages. This leads to unstable insulin dosing, which increases the risk of hypoglycemia.

On the other hand, reducing entropy gradually allows the model to first explore different strategies and then settle into a stable behavior. The ER-PPO model follows this approach, which is why it is able to maintain hypoglycemia at only 1.50%.

So, in simple terms, better control over exploration leads to safer insulin decisions.

4.3.2 Off-Policy Control Requires Explicit Safety Architecture

The TD3 results clearly show that good performance in reinforcement learning does not automatically mean it is safe for medical use.

Even though TD3 is known to be stable and sample-efficient, in this case it produced extremely high insulin doses. This happened because the reward function and action range were not strictly constrained.

As a result, the model learned a harmful strategy — it keeps glucose very low to avoid hyperglycemia, but this leads to severe hypoglycemia.

This highlights an important point: For healthcare applications, safety constraints are not optional, they are necessary.

4.3.3 Rate-of-Change Features Improve Anticipatory Dosing

Another important observation is the role of ΔBG (change in glucose levels).

Including this feature helps the model understand not just the current glucose value, but also whether it is increasing or decreasing. This gives the model a sense of direction, which is very useful.

Because of this, ER-PPO is able to act earlier and prevent large glucose spikes. In contrast, PPO and TD3 react only after glucose has already increased, which results in delayed and sometimes excessive insulin dosing.

So, even a small addition in state representation can significantly improve performance.

4.3.4 ER-PPO Satisfies All Primary Clinical Objectives

ER-PPO is the only model in this study that meets all the important clinical requirements at the same time.

- TIR above 75% → achieved 81.60%
- Hypoglycemia below 4% → achieved 1.50%
- Realistic insulin usage → 30.67 U

Other models either fail in safety (TD3) or consistency (PPO).

Another important point is that ER-PPO achieves this without tuning for individual patients. This is useful because in real-world deployment, training separate models for each patient is not practical.

4.3.5 Supervised Learning Provides a Necessary Prediction Baseline

The results from Module 1 show that TFT performs very well in predicting insulin dosage, with MAE of 0.3954 U.

Even though this model is not directly used in the RL system, it still plays an important role. It shows that the data contains meaningful patterns and that insulin requirements can be estimated accurately.

This also helps in designing reward functions and understanding expected insulin behavior.

So, supervised learning and reinforcement learning are not competing approaches — they complement each other.

CHAPTER 5

Conclusion and Future Work

5.1 Conclusion

This project focused on designing and evaluating adaptive insulin dosage prediction and control systems for Type-1 Diabetes mellitus, moving step by step through three modules with increasing complexity and more realistic conditions. The main goal was to overcome the limitation of traditional insulin delivery systems, which usually cannot adapt properly to sudden changes in glucose levels or unpredictable meal patterns.

In Module 1, three deep learning models — LSTM, BiLSTM, and Temporal Fusion Transformer (TFT) — were used for predicting next-hour insulin dosage using the AIM-94 dataset. Among them, TFT performed the best, achieving a Mean Absolute Error of 0.3954 U, which is clearly better than LSTM and BiLSTM. This shows that attention-based models are more capable in handling complex time-series data. But still, these models work in an open-loop way, so once prediction is done, there is no correction based on what actually happens in the patient.

Module 2 introduced reinforcement learning for closed-loop insulin delivery. The PPO agent was trained using the UVA/Padova simglucose simulator and achieved 86.55% Time-in-Range under fixed meal conditions. This is an important result because it shows that RL can actually learn useful insulin control policies and even cross the clinical benchmark.

However, since the meals were fixed, the environment was too simple, and the model kind of learned patterns instead of real adaptation.

Module 3 is where the main contribution comes. A comparison between PPO, TD3, and ER-PPO was done under random meal conditions, which are more realistic. The ER-PPO model, which uses a decaying entropy from 0.05 to 0.001, achieved 81.60% TIR with only 1.50% hypoglycemia. In comparison, PPO dropped to 65.36% TIR and TD3 performed very poorly in terms of safety.

One important understanding from this work is that exploration in reinforcement learning is not just a training detail, it directly affects patient safety. If the model

keeps exploring too much, it becomes unstable. If it stops exploring too early, it may miss better strategies. ER-PPO balances this properly by reducing exploration gradually.

Overall, this project shows that entropy-regulated reinforcement learning can be a strong approach for insulin control. It connects prediction-based models and real-time control systems in a more practical way. While still in simulation, the results indicate that such methods can be useful for future intelligent healthcare systems.

5.2 Future Study

Even though the results are promising, there are still several areas where improvements can be done. Some of them are listed below.

5.2.1 Multi-Patient Generalization

Right now, the ER-PPO model is tested only on one virtual patient (adult#001). This is not enough to say it will work for everyone.

In future, the model should be tested on multiple patients with different characteristics like body weight, insulin sensitivity, and daily patterns.

Also, methods like meta-learning (for example MAML) can be used so that one model can quickly adapt to new patients without full retraining.

5.2.2 Non-Linear Entropy Decay Schedules

In this project, entropy is reduced linearly, which works fine, but it may not be the best possible approach.

Other methods like exponential decay or cosine scheduling can be explored. Even better, entropy can be adjusted dynamically based on performance (like TIR during training), so the model learns when to explore and when to stabilize.

5.2.3 Integration of Continuous Glucose Monitor Noise

The current simulation environment assumes perfect glucose readings, which is not realistic.

In real-world scenarios, CGM sensors have noise, delays, and sometimes missing values. So future work should include:

- Measurement noise
- Sensor drift
- Missing data

This will make the model more robust and closer to real-world usage.

5.2.4 Incorporation of Exercise and Illness Disturbances

This work mainly focuses on meal disturbances, but in real life, glucose levels are also affected by exercise, stress, and illness.

Adding these factors into the simulation will make the system more realistic and challenging. It will also test whether the model can adapt to sudden unexpected changes.

5.2.5 Safe Reinforcement Learning Constraints

Currently, safety is handled indirectly through reward design and action limits.

In future, more formal safety methods can be used, such as:

- Constrained Markov Decision Processes (CMDP)
- Lyapunov-based safe RL

These methods can ensure that the system never violates safety limits, which is very important in medical applications.

5.2.6 Transfer to Real Patient Data

Finally, the most important step is testing on real patient data.

The model can first be trained in simulation and then fine-tuned using real datasets like OhioT1DM.

This will help in understanding how well the model performs outside simulation.

In the long term, clinical trials will be needed, but that requires strict validation and regulatory approval.

References

[1] Mughal, I. S., Patanè, L., & Caponetto, R. (2024). A comprehensive review of models and nonlinear control strategies for blood glucose regulation in artificial pancreas. *Annual Reviews in Control*, *57*, 100937.

[2] Khanna, A., Francon, O., & Miikkulainen, R. (2024). Optimizing the Design of an Artificial Pancreas to Improve Diabetes Management. *arXiv preprint arXiv:2402.07949*.

[3] Zeng, H., Ji, H., & Zhou, P. (2024). DIETS: Diabetic Insulin Management System in Everyday Life. *arXiv preprint arXiv:2411.12812*.

[4] Milton, T., & Lieck, R. (2024, December). Fully-Automated Patient-Agnostic Diabetes Management with Deep Reinforcement Learning. In *2024 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)* (pp. 1085-1091). IEEE.

[5] Addissouky, T. A., Ali, M. M., El Sayed, I. E. T., & Wang, Y. (2024). Type 1 diabetes mellitus: retrospect and prospect. *Bulletin of the National Research Centre*, *48*(1), 42.

[6] Lu, B., Cui, Y., Belsare, P., Stanger, C., Zhou, X., & Prioleau, T. (2024). Mealtime prediction using wearable insulin pump data to support diabetes management. *Scientific Reports*, *14*(1), 21013.

[7] Jia, W., & Xu, H. (2025). Toward fully stabilized flow control using deep reinforcement learning. *Physics of Fluids*, *37*(7).

[8] Nigi, L., Simon Batzibal, M. D. L. A., Cataldo, D., & Dotta, F. (2024). 12-month time in tight range improvement with advanced hybrid-closed loop system in adults with type 1 diabetes. *Diabetes Therapy*, *15*(12), 2557-2568.

[9] Mujahid, O., Contreras, I., Beneyto, A., & Vehi, J. (2024). Generative deep learning for the development of a type 1 diabetes simulator. *Communications Medicine*, *4*(1), 51.

[10] Zhao, Y. F., Chaw, J. K., Ang, M. C., Tew, Y., Shi, X. Y., Liu, L., & Cheng, X. (2025). A safe-enhanced fully closed-loop artificial pancreas controller based on deep reinforcement learning. *PloS one*, *20*(1), e0317662.

[11] Chu, S. J., Amarasiri, N., Giri, S., & Kafle, P. (2025). Blood glucose level prediction in type 1 diabetes using machine learning. *arXiv preprint arXiv:2502.00065*.

[12] Ganji, M., El Fathi, A., Fabris, C., Lv, D., Kovatchev, B., & Breton, M. (2025). Distribution-based sub-population selection (DSPS): A method for in-silico reproduction of clinical trials outcomes. *Computers in Biology and Medicine*, *186*, 109714.

[13] Zhu, T., Kuang, L., Piao, C., Zeng, J., Li, K., & Georgiou, P. (2024). Population-specific glucose prediction in diabetes care with transformer-based deep learning on the edge. *IEEE transactions on biomedical circuits and systems*, *18*(2), 236-246.

[14] Desman, J. M., Hong, Z. W., Sabounchi, M., Sawant, A. S., Gill, J., Costa, A. C., ... & Sakhuja, A. (2025). A distributional reinforcement learning model for optimal glucose control after cardiac surgery. *npj Digital Medicine*, *8*(1), 313.

[15] Sun, Q., Jankovic, M. V., Budzinski, J., Moore, B., Diem, P., Stettler, C., & Mougiakakou, S. G. (2018). A dual mode adaptive basal-bolus advisor based on reinforcement learning. *IEEE journal of biomedical and health informatics*, *23*(6), 2633-2641.

[16] Wang, G., Liu, X., Ying, Z., Yang, G., Chen, Z., Liu, Z., ... & Chen, Y. (2023). Optimized glycemic control of type 2 diabetes with reinforcement learning: a proof-of-concept trial. *Nature Medicine*, *29*(10), 2633-2642.

[17] Jafar, A., Kobayati, A., Tsoukas, M. A., & Haidar, A. (2024). Personalized insulin dosing using reinforcement learning for high-fat meals and aerobic exercises in type 1 diabetes: a proof-of-concept trial. *Nature Communications*, *15*(1), 6585.

[18] Symeonidis, P., Rizos, E., Andras, C., Hairistanidis, S., Manolopoulos, Y., & Zanker, M. (2025). Deep reinforcement learning for personalized insulin dosing and glucose control of hospitalized in ICU patients. *International Journal of Data Science and Analytics*, *20*(7), 6841-6854.

[19] Fox, I., Lee, J., Pop-Busui, R., & Wiens, J. (2020, September). Deep reinforcement learning for closed-loop blood glucose control. In *Machine Learning for Healthcare Conference* (pp. 508-536). PMLR.

[20] Javad, M. O. M., Agboola, S. O., Jethwani, K., Zeid, A., & Kamarthi, S. (2019). A reinforcement learning–based method for management of type 1 diabetes: exploratory study. *JMIR diabetes*, *4*(3), e12905.

[21] Dénes-Fazakas, L., Fazakas, G. D., Eigner, G., Kovács, L., & Szilágyi, L. (2024, May). Review of reinforcement learning-based control algorithms in artificial pancreas systems for diabetes mellitus management. In *2024 IEEE 18th International Symposium on Applied Computational Intelligence and Informatics (SACI)* (pp. 000565-000572). IEEE.

[22] Emerson, H., Guy, M., & McConville, R. (2023). Offline reinforcement learning for safer blood glucose control in people with type 1 diabetes. Journal of Biomedical Informatics, 142, 104376. doi.org

[23] Kumar, A., & Singh, D. (2026). Deep-Q-Network in Multiview Ensemble Learning to Predict Anti-diabetic Peptide and Diabetic Types. *IEEE Transactions on Computational Biology and Bioinformatics*.

[24] Lee, S., Kim, J., Park, S. W., Jin, S. M., & Park, S. M. (2020). Toward a fully automated artificial pancreas system using a bioinspired reinforcement learning design: In silico validation. *IEEE Journal of Biomedical and Health Informatics*, *25*(2), 536-546.

[25] Seo, Y., Sferrazza, C., Geng, H., Nauman, M., Yin, Z. H., & Abbeel, P. (2025). Fasttd3: Simple, fast, and capable reinforcement learning for humanoid control. *arXiv preprint arXiv:2505.22642*.

[26] Yu, X., Yang, Z., Sun, X., Liu, H., Li, H., Lu, J., ... & Cinar, A. (2025). Deep reinforcement learning for automated insulin delivery systems: algorithms, applications, and prospects. *AI*, *6*(5), 87.

APPENDIX

Some screenshots of the code snippet:

```python
# --- 1. RANDOMIZATION WRAPPER (CORRECTED IMPLEMENTATION) ---
class RandomizedScenarioWrapper(gym.Wrapper):
    def __init__(self, env):
        super().__init__(env)

    def _create_random_scenario(self):
        meal_time = random.randint(30, 120)
        meal_amount = random.randint(30, 100)
        return CustomScenario(start_time=0, scenario=[(meal_time, meal_amount)])

    def reset(self, **kwargs):
        new_scenario = self._create_random_scenario()
        self.env.unwrapped.scenario = new_scenario
        return self.env.reset(**kwargs)
```

Figure 4.7: Randomized Meal Scenario Wrapper

```python
# --- 3. REWARD FUNCTION (WITH BALANCED PENALTIES) ---
def gaussian_reward_shaping(bg_history, **kwargs):
    bg = bg_history[-1]
    mu, sigma, amplitude = 115, 20, 0.05
    reward = amplitude * np.exp(-0.5 * ((bg - mu) / sigma) ** 2)
    scaling_factor = 0.0001

    if bg < 70:
        penalty = 5.0 * ((bg - 70) ** 2)
        return -scaling_factor * penalty
    elif bg > 180:
        hyper_penalty_multiplier = 2.0
        penalty = hyper_penalty_multiplier * ((bg - 180) ** 2)
        return -scaling_factor * penalty
    else:
        return reward
```

Figure 4.8: Reward Function for Blood Glucose Control

```python
# --- 2. FEATURE ENGINEERING WRAPPER  ---
class SimglucoseAdvantageWrapper(gym.Wrapper):
    def __init__(self, env, action_penalty_weight=0.01):
        super().__init__(env)
        self.action_penalty_weight = action_penalty_weight
        self.last_action = 0.0
        self.observation_space = gym.spaces.Box(
            low=np.array([0.0, 0.0, 0.0]),
            high=np.array([1000.0, 100.0, 300.0]),
            dtype=np.float32
        )

    def _augment_observation(self, obs, info):
        iob = info.get('IOB', 0.0)
        cob = info.get('CHO', 0.0)
        return np.array([obs[0], iob, cob], dtype=np.float32)

    def reset(self, **kwargs):
        obs, info = self.env.reset(**kwargs)
        self.last_action = 0.0
        return self._augment_observation(obs, info), info

    def step(self, action):
        obs, reward, terminated, truncated, info = self.env.step(action)
        augmented_obs = self._augment_observation(obs, info)
        action_penalty = self.action_penalty_weight * ((action[0] - self.last_action)**2)
        final_reward = reward - action_penalty
        self.last_action = action[0]
        return augmented_obs, final_reward, terminated, truncated, info
```

Figure 4.9: State Feature Engineering for RL Agent

```python
# --- 5. ENVIRONMENT CREATOR ---
def make_env(patient_name_list):
    def _init():
        patient_name = random.choice(patient_name_list)

        if 'simglucose-v0' not in gym.envs.registry:
            gym.register(id='simglucose-v0', entry_point='simglucose.envs.simglucose_gym_env:T1DSimGymnaisumEnv')

        env = gym.make('simglucose-v0', patient_name=patient_name, reward_fun=gaussian_reward_shaping, max_episode_steps=1000)

        env = RandomizedScenarioWrapper(env)
        env = SimglucoseAdvantageWrapper(env)
        env = Monitor(env)
        return env
    return _init
```

Figure 4.10: RL Environment Setup and Integration Pipeline

C-5.pdf

ATLAS SkillTech University

Document Details

Submission ID
trn:oid:::3618:135741054

Submission Date Apr 19, 2026,
5:59 PM GMT+5:30

Download Date Apr 19, 2026,
6:00 PM GMT+5:30

File Name
C-5.pdf

File Size
2.5 MB

76 Pages

14,511 Words

82,026 Characters

7%Overall Similarity

Thecombinedtotalofallmatches,includingoverlappingsources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text

Match Groups

157Not Cited or Quoted 7% Matches with neither in-text citation nor quotation marks

0Missing Quotations 0%
Matches that are still very similar to source material

0Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

5% Internet sources

3% Publications

10% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags forReview

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.
A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

C-5.pdf

ATLAS SkillTech University

Document Details

Submission ID
trn:oid:::3618:135741054

Submission Date Apr 19, 2026,
5:59 PM GMT+5:30

Download Date Apr 19, 2026,
6:00 PM GMT+5:30

File Name
C-5.pdf

File Size
2.5 MB

76 Pages

14,511 Words

82,026 Characters

*% detected as AI

AIdetectionincludesthe possibility of falsepositives.Although some text in thissubmissionis likely AI generated, scoresbelowthe 20% threshold are not surfaced because they have a higher likelihoodoffalse positives.

Caution: Review required.
It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AIwriting assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

Howshould I interpretTurnitin'sAIwritingpercentageandfalsepositives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

Whatdoes'qualifyingtext'mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.